

# Pre-processing before and during training

*Image data, class 2 of 2. Classical operations, normalization done without leaking, medical intensity standardization, and augmentation from flips to MixUp.*

Asst. Prof. Thanapong Intharah, PhD

Visual Intelligence Laboratory · Khon Kaen University

A curated compilation of openly published university teaching materials. Every section names its source; original slides are shown unmodified.

# Today, in three hours

|           |                                                                                                                    |                                 |
|-----------|--------------------------------------------------------------------------------------------------------------------|---------------------------------|
| 0:00–0:25 | <b>D1 · Filtering as preprocessing.</b> Point ops, convolution, Gaussian vs box, sharpening restraint.             | CMU 16-385, Lecture 2           |
| 0:25–0:45 | <b>D2 · Medical classical ops.</b> Histograms on CT, median filtering, MRI denoising, SNR and CNR.                 | Bagci Lab BME 495, Lectures 3–4 |
| 0:45–1:00 | <b>D3 · Contrast, color, crop.</b> Histogram equalization and CLAHE, conversions, cropping and padding.            | authored                        |
| 1:00–1:10 | <i>Break</i>                                                                                                       |                                 |
| 1:10–1:30 | <b>E1 · Preprocessing for the model.</b> Zero-center, normalize, per-channel statistics, and why optimizers care.  | UMich EECS 498, Lecture 10      |
| 1:30–1:50 | <b>E2 · The rule + medical standardization.</b> Statistics from training data only; MRI intensity standardization. | authored + Bagci L3–4           |
| 1:50–2:05 | <b>E3 · Transfer learning.</b> Pretrained backbones and the statistics they expect.                                | Stanford CS231n, Lecture 7      |
| 2:05–2:40 | <b>F1–F2 · Augmentation.</b> Flips to MixUp; medical augmentation incl. scanner artifacts.                         | EECS 498 L10 + Bagci L9         |
| 2:40–3:00 | <b>F3 · Pipelines in code.</b> torchvision v2, Albumentations, Keras layers, MONAI, CutMix in the loader.          | authored                        |

# Filtering: the classical preprocessing toolbox

*Point operations, convolution, and smoothing. The same kernels you tune by hand here are what a CNN's first layer learns for itself, which is why this vocabulary matters.*

# Image filtering



# Linear shift-invariant image filtering

- Replace each pixel by a *linear* combination of its neighbors (and possibly itself).
- The combination is determined by the filter's *kernel*.
- The same kernel is *shifted* to all pixel locations so that all pixels use the same linear combination of their neighbors.

# Example: the box filter

- also known as the 2D rect (not rekt) filter
- also known as the square mean filter

kernel  $g[\cdot, \cdot] = \frac{1}{9}$

|   |   |   |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 1 | 1 |
| 1 | 1 | 1 |

Handwritten note: 1x3x3

- replaces pixel with local average
- has smoothing (blurring) effect

Handwritten note: ฟิลเตอร์ Blur



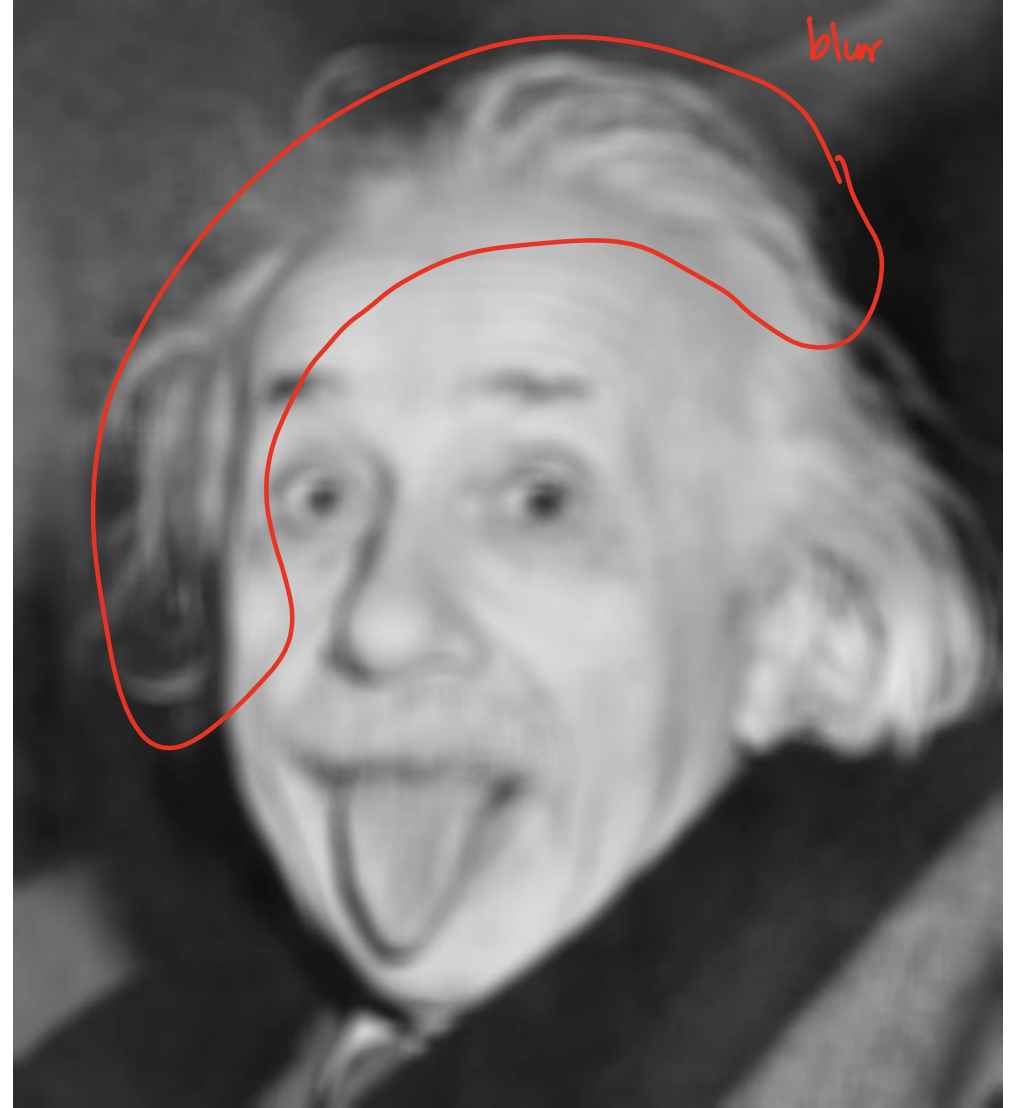
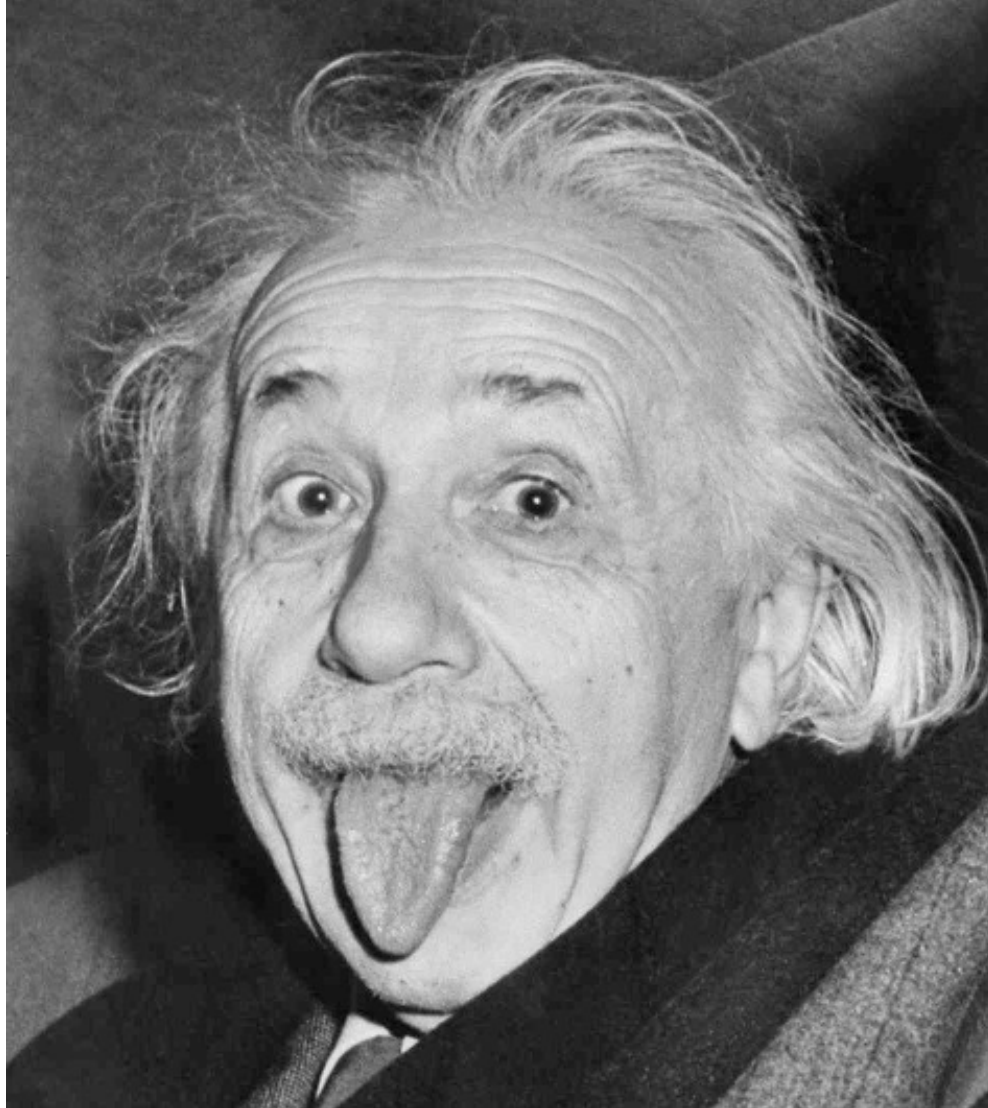
# ... and the result is



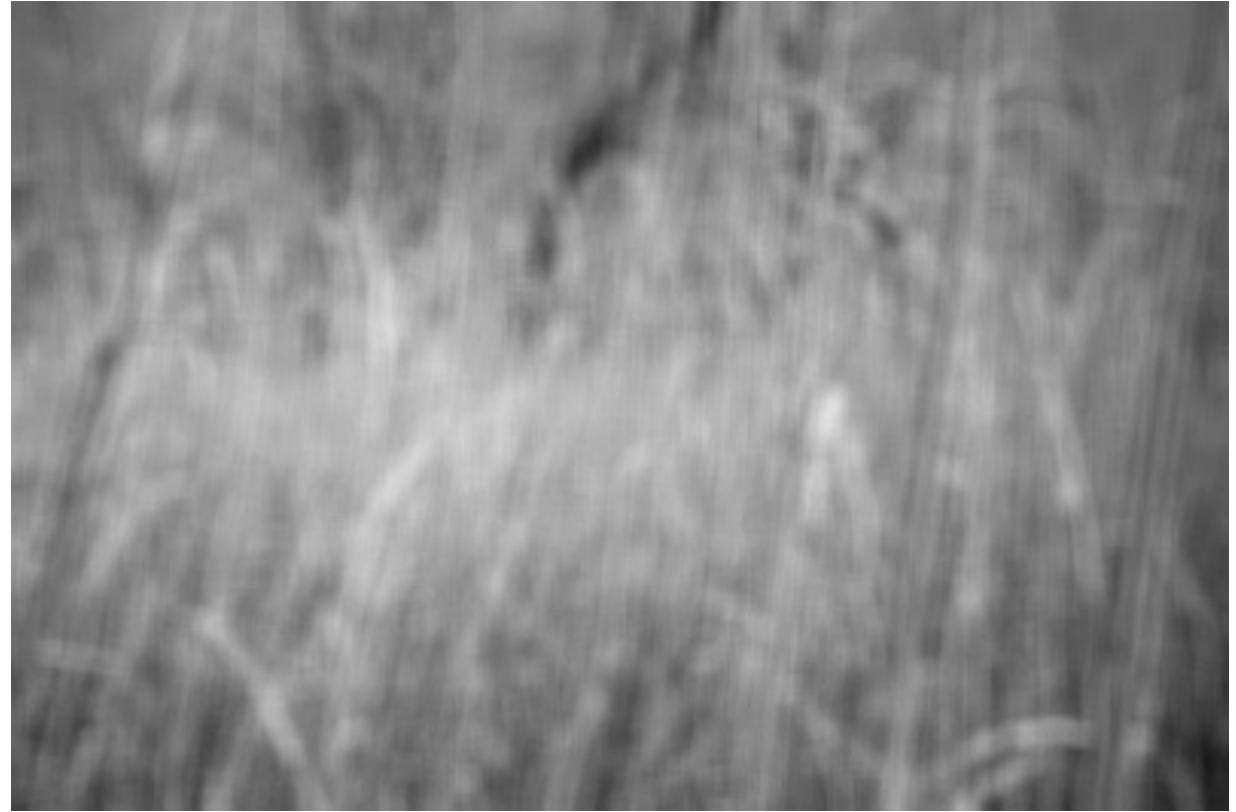
$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output  
 filter  
 image (signal)

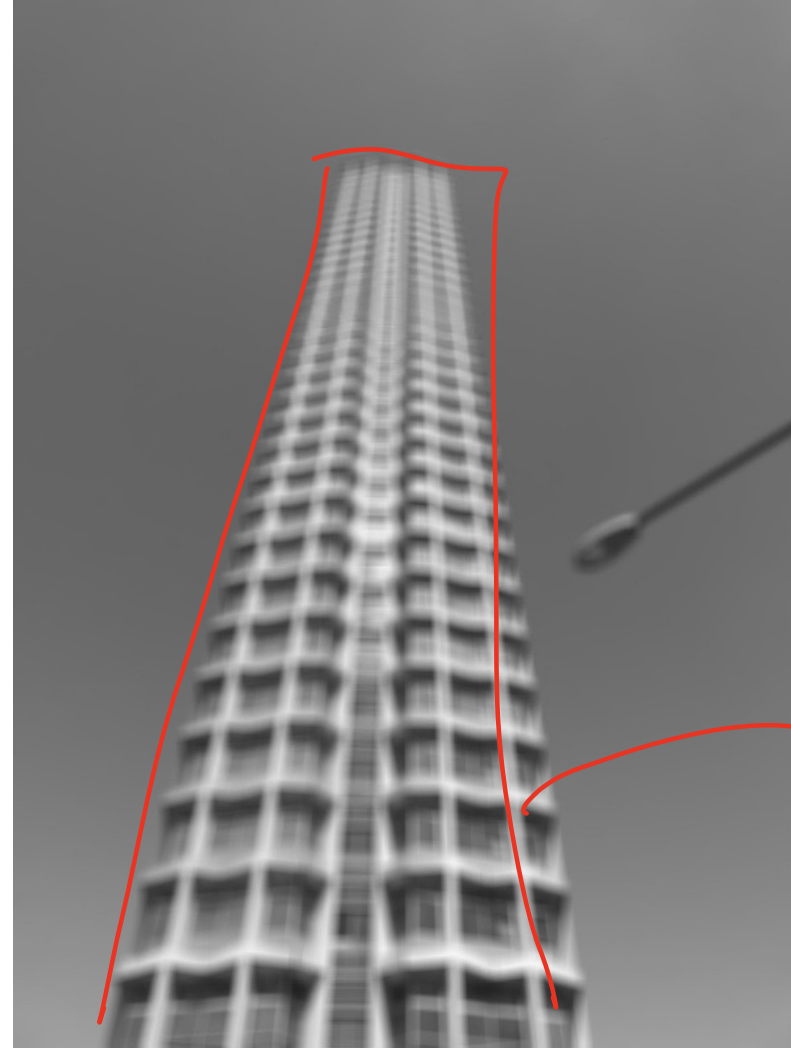
# Some more realistic examples



# Some more realistic examples



# Some more realistic examples



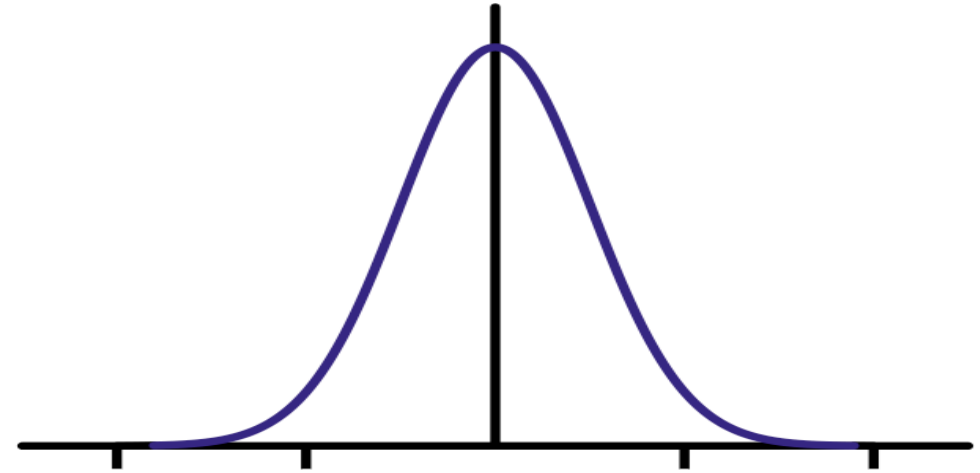
blur

# The Gaussian filter

- named (like many other things) after Carl Friedrich Gauss
- kernel values sampled from the 2D Gaussian function:  
$$f(i, j) = \frac{1}{2\pi\sigma^2} e^{-\frac{i^2+j^2}{2\sigma^2}}$$
Normal
- weight falls off with distance from center pixel
- theoretically infinite, in practice truncated to some maximum distance

Any heuristics for selecting where to truncate?

- usually at  $2-3\sigma$



Is this a separable filter? Yes!

kernel  $\frac{1}{16}$

|   |   |   |
|---|---|---|
| 1 | 2 | 1 |
| 2 | 4 | 2 |
| 1 | 2 | 1 |

Gaussian Filter

$$f(i, j) = \frac{1}{2\pi\sigma^2} e^{-\frac{i^2+j^2}{2\sigma^2}}$$

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

$(i^2+j^2)$

$$\begin{bmatrix} (-1,-1) & (-1,0) & (-1,1) \\ (0,-1) & (0,0) & (0,1) \\ (1,-1) & (1,0) & (1,1) \end{bmatrix}$$



$$\begin{bmatrix} 2 & 1 & 2 \\ 1 & 0 & 1 \\ 2 & 1 & 2 \end{bmatrix}$$

$$\frac{f(i=1, j=0)}{f(i=0, j=0)} = \frac{1}{2\pi\sigma^2} e$$

$$2\sigma^2 = \frac{1}{\ln 2} \quad (\sigma = 0.8493)$$

นิยาม

$$e^{-\frac{i^2+j^2}{2\sigma^2}} = 2^{-(i^2+j^2)}$$

(ค่าของ  $i, j$  1:2:4)

$$\begin{bmatrix} 0.25 & 0.50 & 0.25 \\ 0.50 & 1 & 0.50 \\ 0.25 & 0.50 & 0.25 \end{bmatrix} \propto 4$$

$$= \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

$$1+2+1+\dots+2+1 = 16$$

$$f(i, j) = \frac{1}{2\pi\sigma^2} e^{-\frac{i^2+j^2}{2\sigma^2}}$$

$$\frac{1}{16}$$

|   |   |   |
|---|---|---|
| 1 | 2 | 1 |
| 2 | 4 | 2 |
| 1 | 2 | 1 |

$$i, j = \{-2, -1, 0, 1, 2\}$$

$$\sigma^2 = 1$$

$$\frac{1}{2\pi} e^{-\frac{(i^2+j^2)}{2}}$$

|         |         |        |        |        |
|---------|---------|--------|--------|--------|
| (-2,2)  | (-1,2)  | (0,2)  | (1,2)  | (2,2)  |
| (-2,1)  | (-1,1)  | (0,1)  | (1,1)  | (2,1)  |
| (-2,0)  | (-1,0)  | (0,0)  | (1,0)  | (2,0)  |
| (-2,-1) | (-1,-1) | (0,-1) | (1,-1) | (2,-1) |
| (-2,-2) | (-1,-2) | (0,-2) | (1,-2) | (2,-2) |

5x5

=

|        |        |        |        |        |
|--------|--------|--------|--------|--------|
| 0.0183 | 0.0821 | 0.1353 | 0.0821 | 0.0183 |
| 0.0821 | 0.3679 | 0.6065 | 0.3679 | 0.0821 |
| 0.1353 | 0.6065 | 1      | 0.6065 | 0.1353 |
| 0.0821 | 0.3679 | 0.6065 | 0.3679 | 0.0821 |
| 0.0183 | 0.0821 | 0.1353 | 0.0821 | 0.0183 |

5x5

●  $i^2+j^2 = 0$  ;  $e^0 = 1$

●  $i^2+j^2 = 1$  ;  $e^{-1/2} = 0.6065$

●  $i^2+j^2 = 2$  ;  $e^{-1} = 0.3679$

●  $i^2+j^2 = 5$  ;  $e^{-5/2} = 0.0821$

●  $i^2+j^2 = 8$  ;  $e^{-8/2} = 0.0183$

●  $i^2+j^2 = 4$  ;  $e^{-4/2} = 0.1353$

$$= \begin{bmatrix} 0.0183 & 0.0821 & 0.1353 & 0.0821 & 0.0183 \\ 0.0821 & 0.3679 & 0.6065 & 0.3679 & 0.0821 \\ 0.1353 & 0.6065 & 1 & 0.6065 & 0.1353 \\ 0.0821 & 0.3679 & 0.6065 & 0.3679 & 0.0821 \\ 0.0183 & 0.0821 & 0.1353 & 0.0821 & 0.0183 \end{bmatrix} \div 0.0183$$

$$= \begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 20 & 33 & 20 & 4 \\ 7 & 33 & 55 & 33 & 7 \\ 4 & 20 & 33 & 20 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix}_{5 \times 5}$$

$$= \frac{1}{339}$$

$$\begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 20 & 33 & 20 & 4 \\ 7 & 33 & 55 & 33 & 7 \\ 4 & 20 & 33 & 20 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix}_{5 \times 5}$$

$$; 1+4+7+\dots+7+4 = 339$$



# Gaussian Filter Visualization (5x5 Integer Kernel, $\sigma^2=1$ )



## $f$ CONTINUOUS FUNCTION

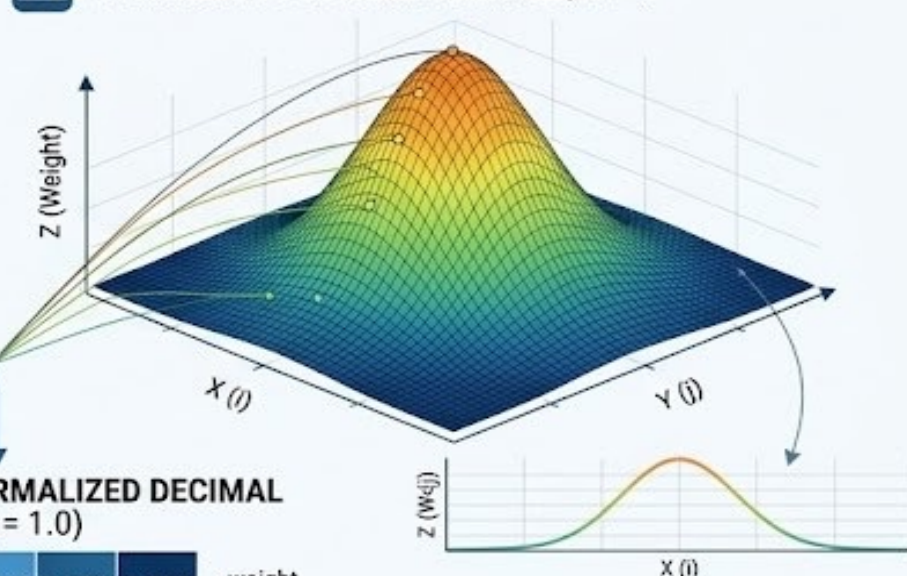
$$f(i, j) = \frac{1}{2\pi\sigma^2} e^{-\frac{i^2+j^2}{2\sigma^2}}$$

$i, j$  = grid coordinates from (-2, -2) to (2, 2)  
 $e$  = natural log  
 $\sigma^2=1$  = variance

## PARAMETER CONTEXT

Size: 5x5 grid  
Mean: (0,0)  
Integer Scaling Factor: 1/331

## CONTINUOUS 3D BELL SURFACE / $\sigma = 1$



## DISCRETE INTEGER WEIGHTS KERNEL (UNNORMALIZED)

|                                |            |                           |                 |    |   |
|--------------------------------|------------|---------------------------|-----------------|----|---|
| OUTER EDGE<br>(e.g., 2,0) = 7  | 1          | 4                         | 7               | 4  | 1 |
| INNER EDGE<br>(e.g., 1,0) = 33 | 4          | 20                        | 33              | 20 | 4 |
|                                | 7          | 33                        | 55              | 33 | 7 |
| OUTER EDGE<br>(e.g., 2,0) = 7  | 4          | 20                        | 33              | 20 | 4 |
| SIDE EDGE = 4                  | 1          | 4                         | 7               | 4  | 1 |
|                                | OUTER EDGE | CENTER PEAK<br>(0,0) = 55 | OUTER CORNE = 4 |    |   |

SUM OF ALL WEIGHTS  
= 1 + 4\*4 + 4\*7 + 8\*4 + 4\*1 = 331  
= 331

INNER CORNER  
(e.g., 1,1) = 20

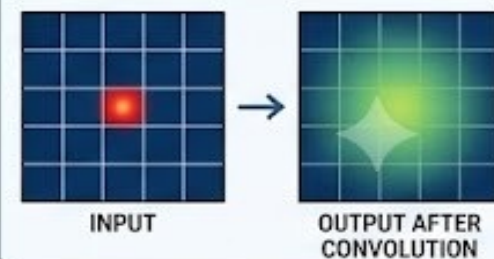
OUTER CORNER  
(e.g., 2,2) = 1

## DISCRETE NORMALIZED DECIMAL KERNEL (SUM = 1.0)

|                   |        |        |        |        |        |                   |
|-------------------|--------|--------|--------|--------|--------|-------------------|
| weight<br>≈ w/331 | 0.0030 | 0.0997 | 0.1662 | 0.0997 | 0.0030 | weight<br>≈ w/331 |
| weight<br>≈ w/331 | 0.0997 | 0.0997 | 0.1662 | 0.0997 | 0.0997 |                   |
| weight<br>≈ w/331 | 0.0997 | 0.1662 | 0.1662 | 0.1997 | 0.0997 | weight<br>≈ w/331 |
| weight<br>≈ w/331 | 0.0997 | 0.0997 | 0.1662 | 0.0997 | 0.0997 |                   |
| weight<br>≈ w/331 | 0.0030 | 0.0030 | 0.0030 | 0.0030 | 0.0030 | weight<br>≈ w/331 |
| TOTAL SUM = 1.000 |        |        |        |        |        |                   |

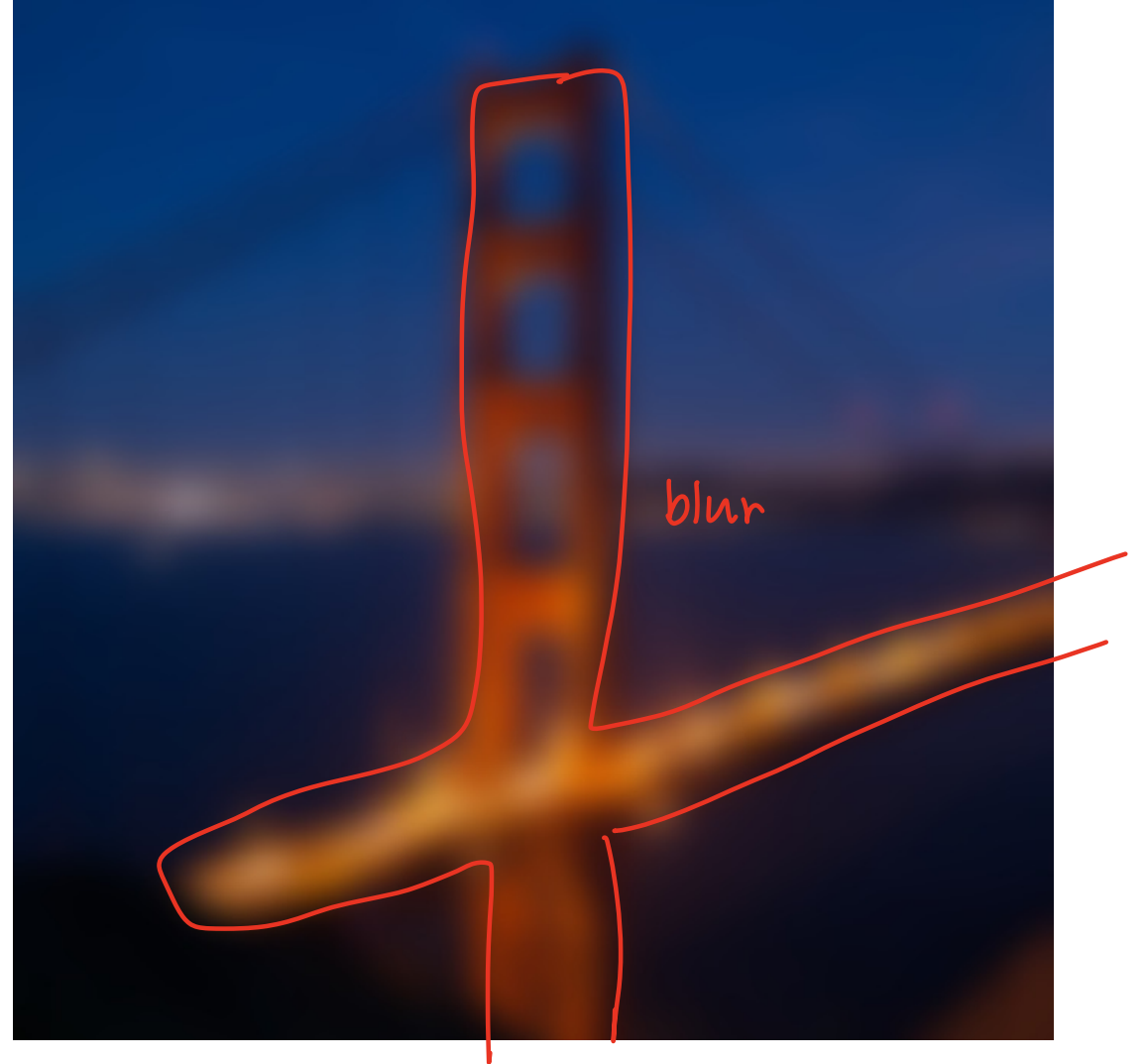
## FILTER APPLICATION EXAMPLE

CONVOLUTION: BLURRING AND SMOOTHING EFFECT

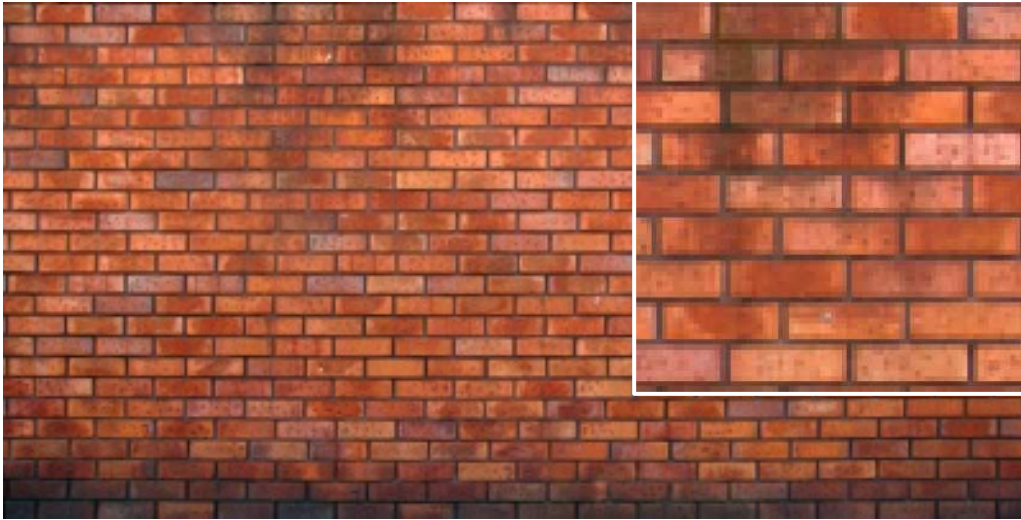


VISUAL AID FOR DISCRETE GAUSSIAN FILTERS

# Gaussian filtering example

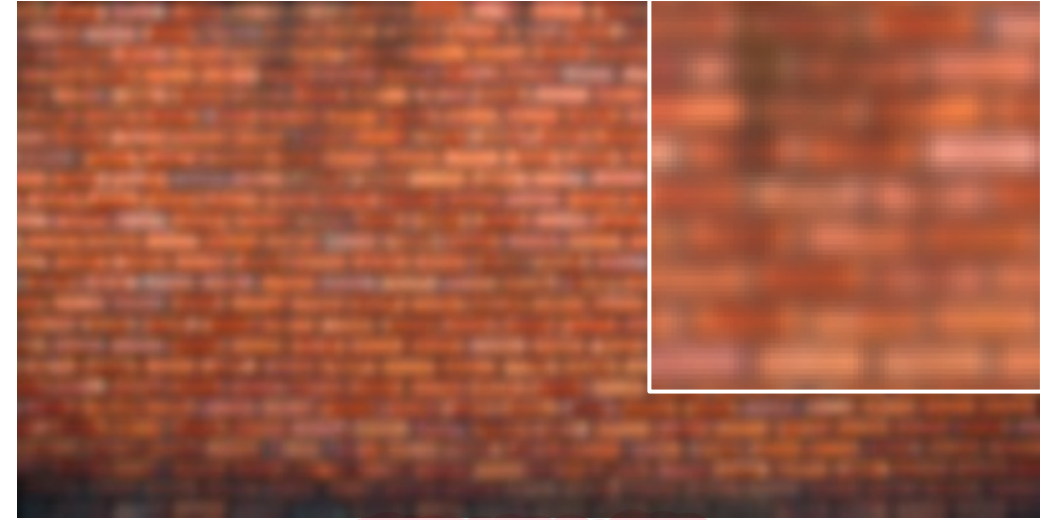


# Gaussian vs box filtering



original

Which blur do you like better?



7x7 Gaussian



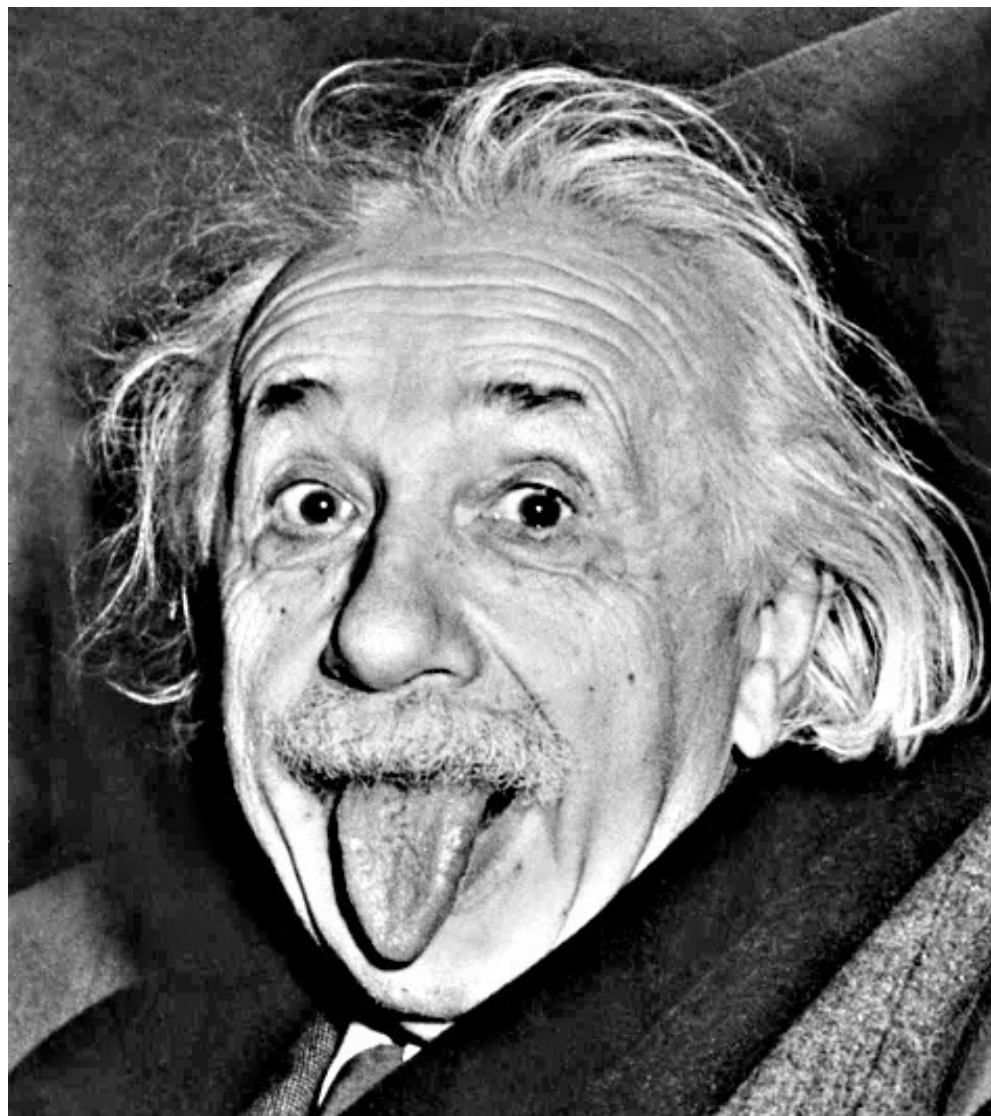
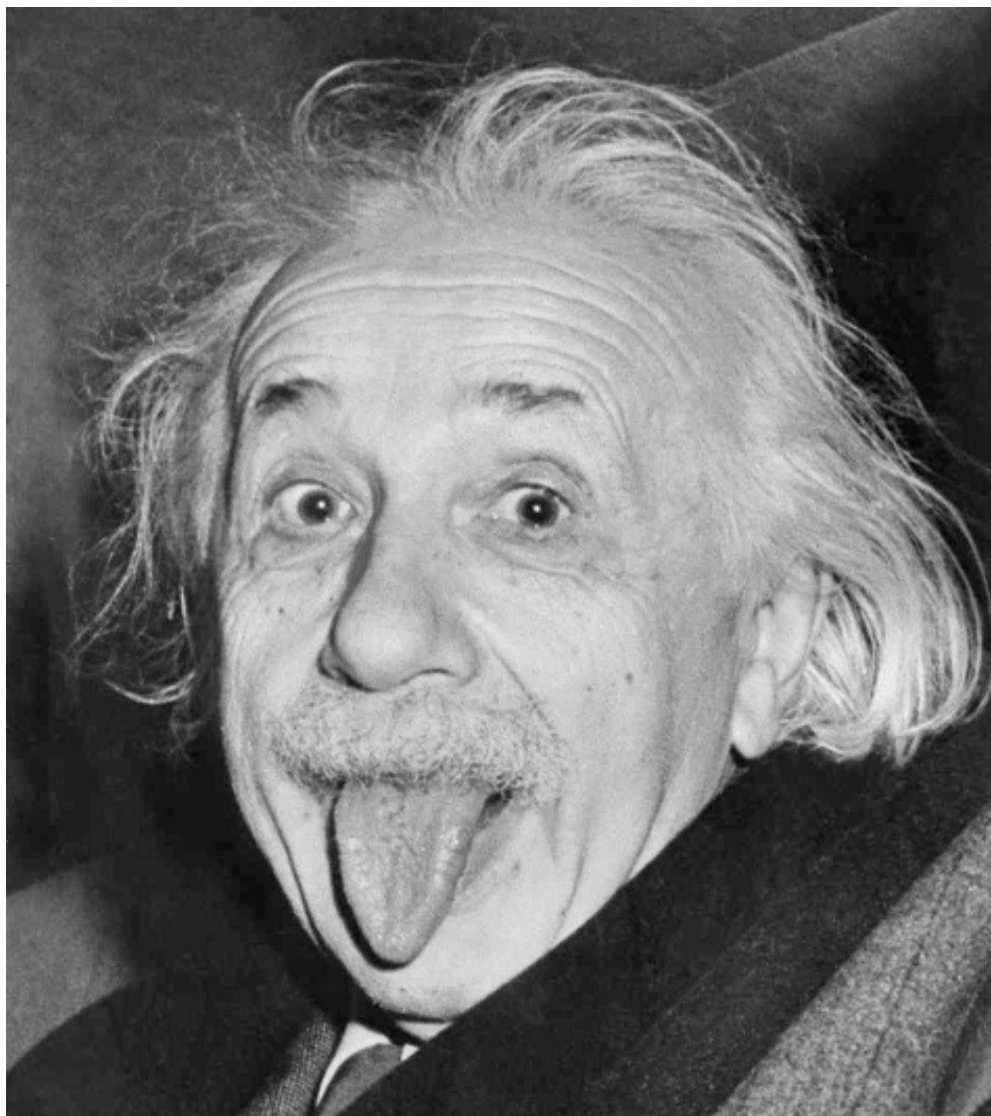
7x7 box

|    |    |    |
|----|----|----|
| -1 | -1 | -1 |
| -1 | 9  | -1 |
| -1 | -1 | -1 |

Sharpen  
filter

# Sharpening examples

มี filter  
ที่ทำให้ภาพชัดได้



# Do not overdo it with sharpening



original



sharpened



oversharpened

What is wrong in this image?

# The same toolbox, on medical images

*Histograms that read lung disease, median filters on X-rays, MRI denoising judged by SNR and CNR: preprocessing with a clinical meaning attached to every intensity.*

SOURCE DECK

**Image preprocessing**, BME 495 Deep Learning for Biomedical Imaging, Winter 2025, Lectures 3–4 · Ulas Bagci, Northwestern University.

<https://bagcilab.com/courses/> publicly shared by the author · shown unmodified · selected pages

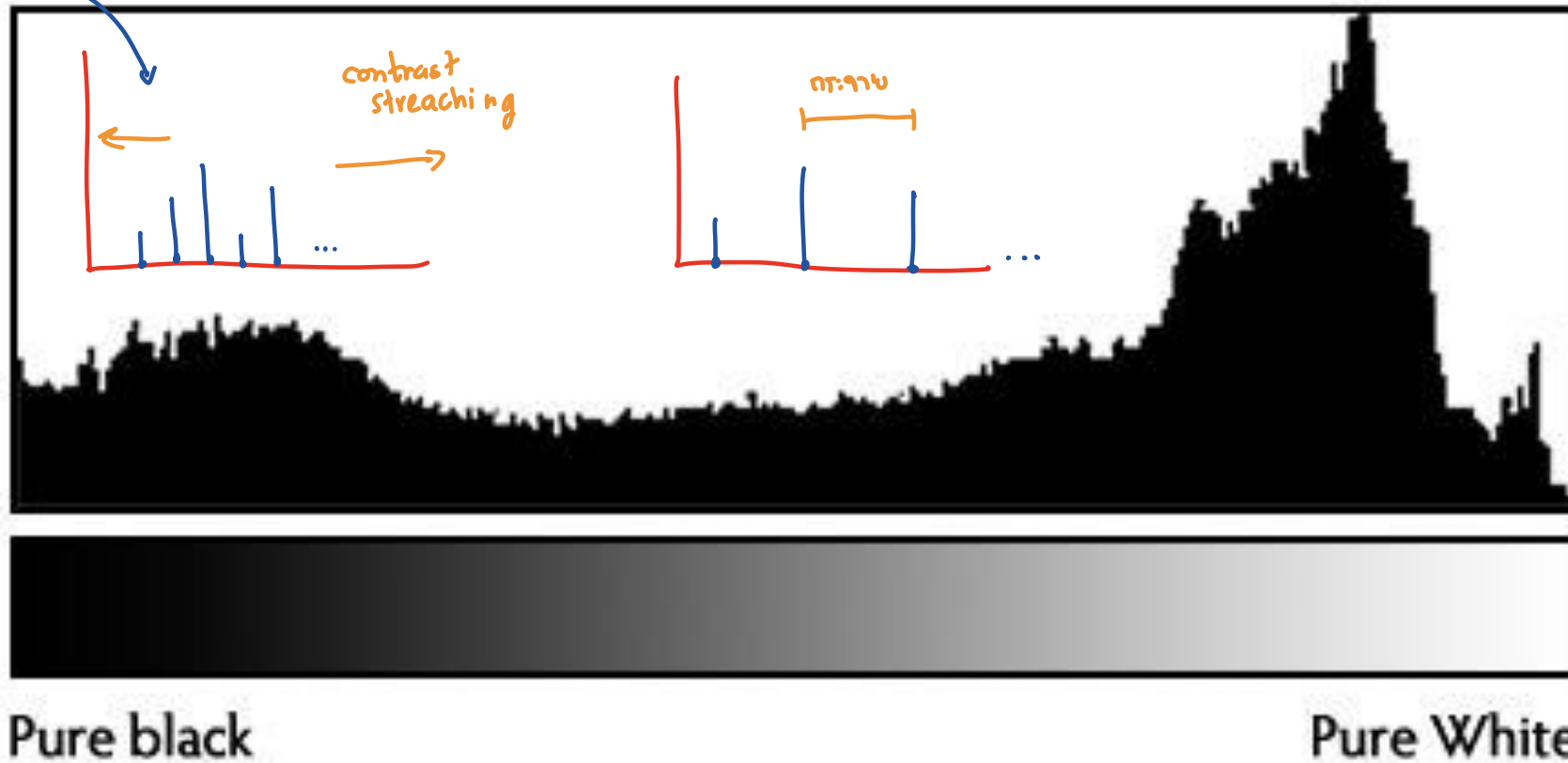
A top-down view of a rectangular box filled with macarons. The macarons are arranged in two rows. The top row contains six macarons: two light green, two light brown, one yellow, one light green, and one pink. The bottom row contains six macarons: two light brown, one yellow, one light green, one pink, and one light green. The macarons have a smooth, slightly domed shape with a visible filling between the two shells. The box is dark brown and the background is a light, neutral color.

# Image Histogram ?

# Image Histogram

|     |     |     |
|-----|-----|-----|
| 100 | 100 | 150 |
| 200 | 2   | 80  |
| 2   | 150 | 80  |

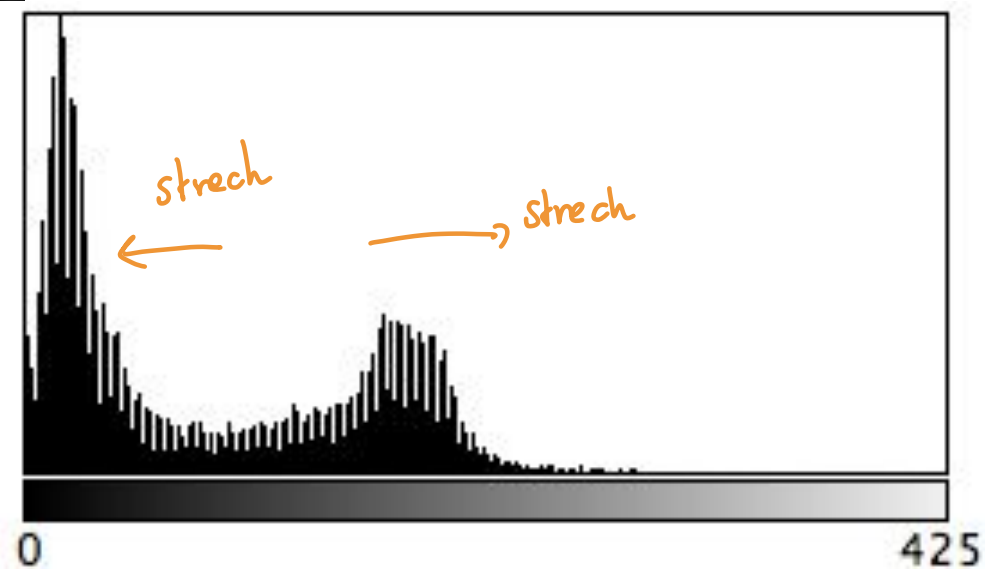
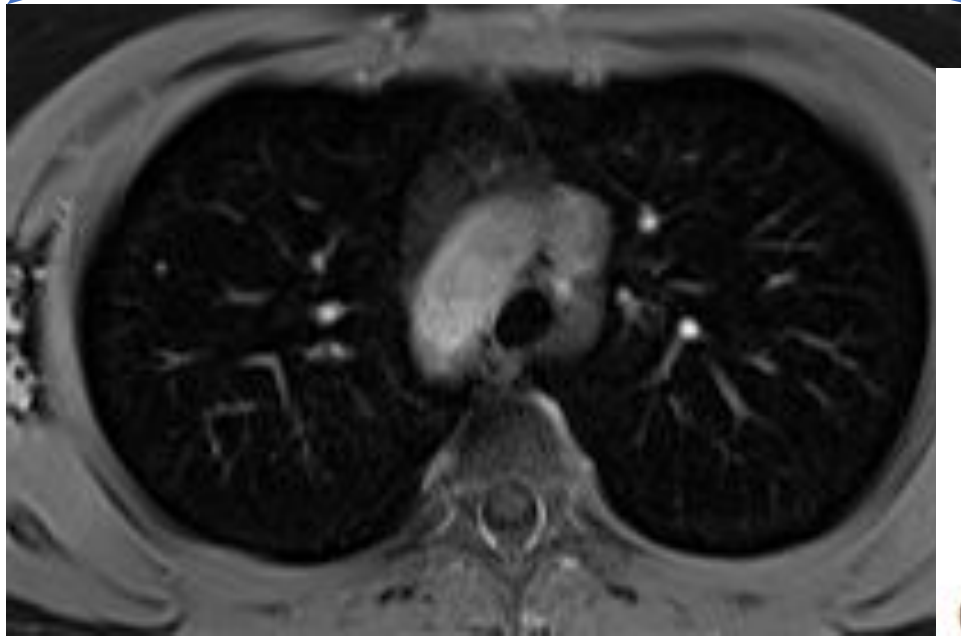
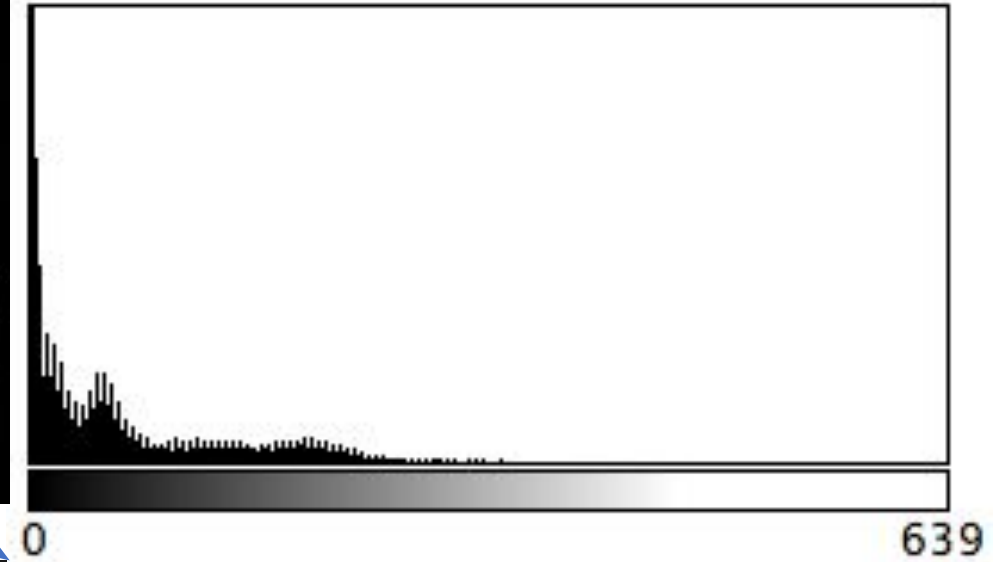
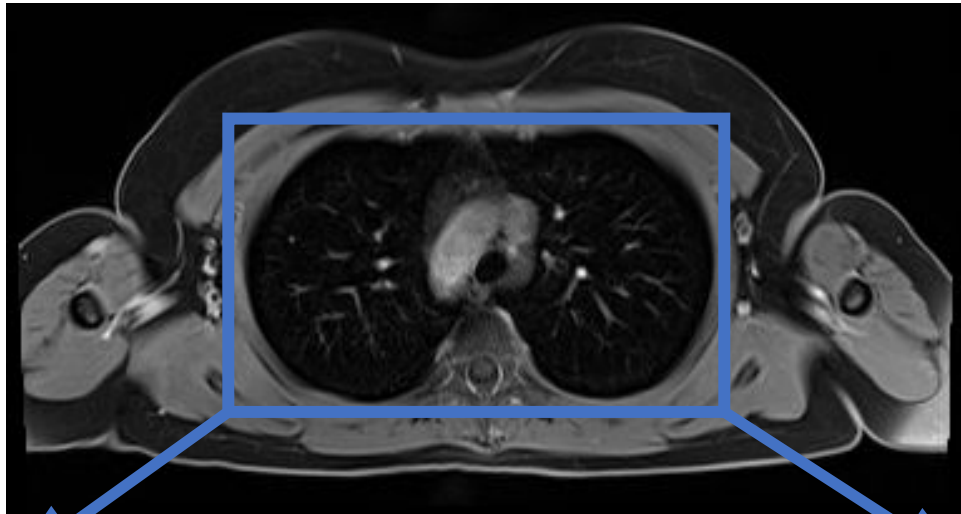
Number of pixels that tone



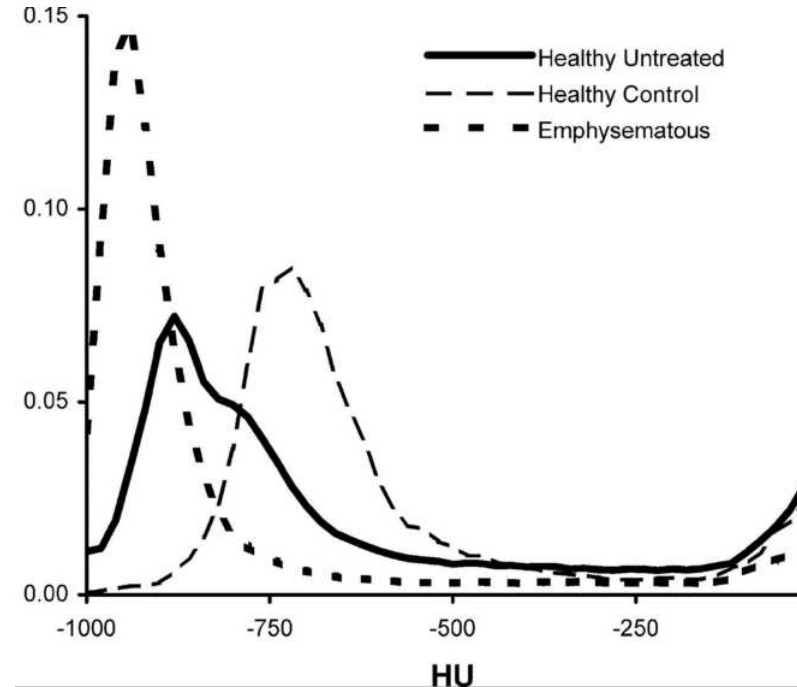
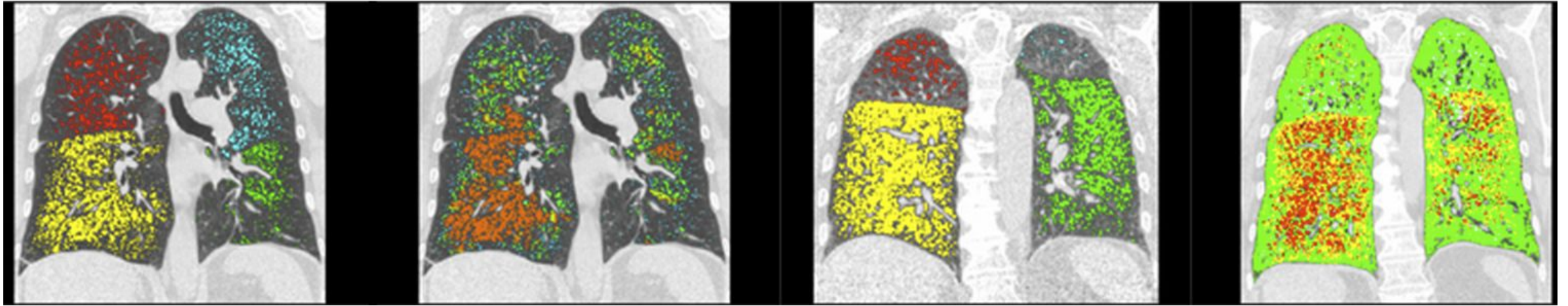
It plots the *number of pixels* for each intensity value.



# Image Histogram – Example



# Histogram based analysis of Lung CT (credit: *Imbio*)



# Median Filters (non-linear)

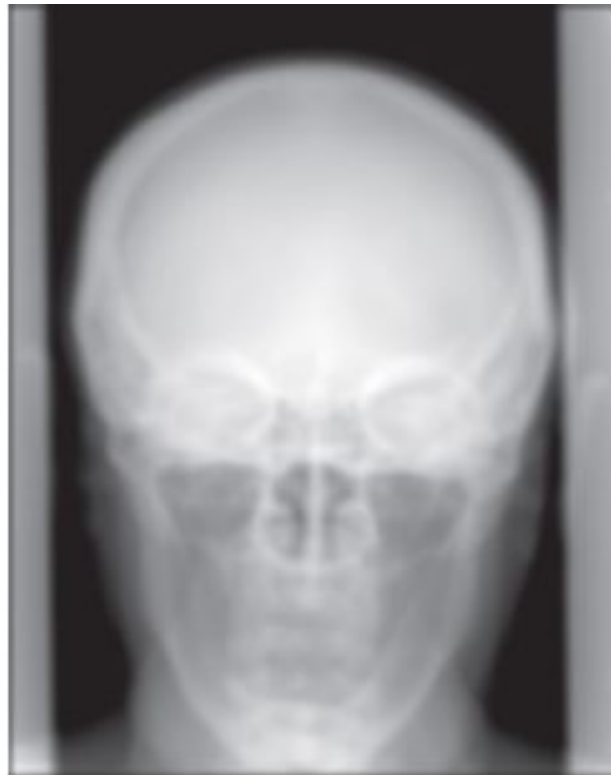
- A **Median Filter** operates over a window by selecting the median intensity in the window. 



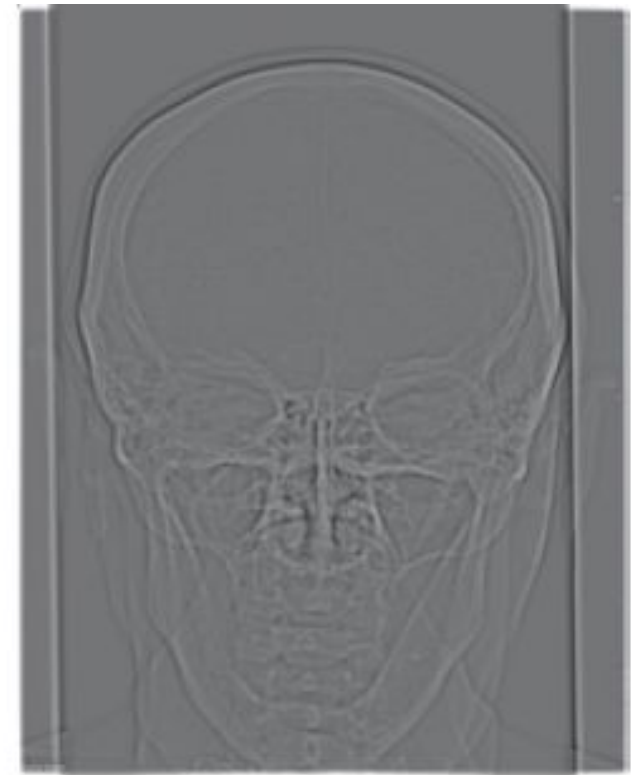
# Filtering X-ray



(a)



(b)

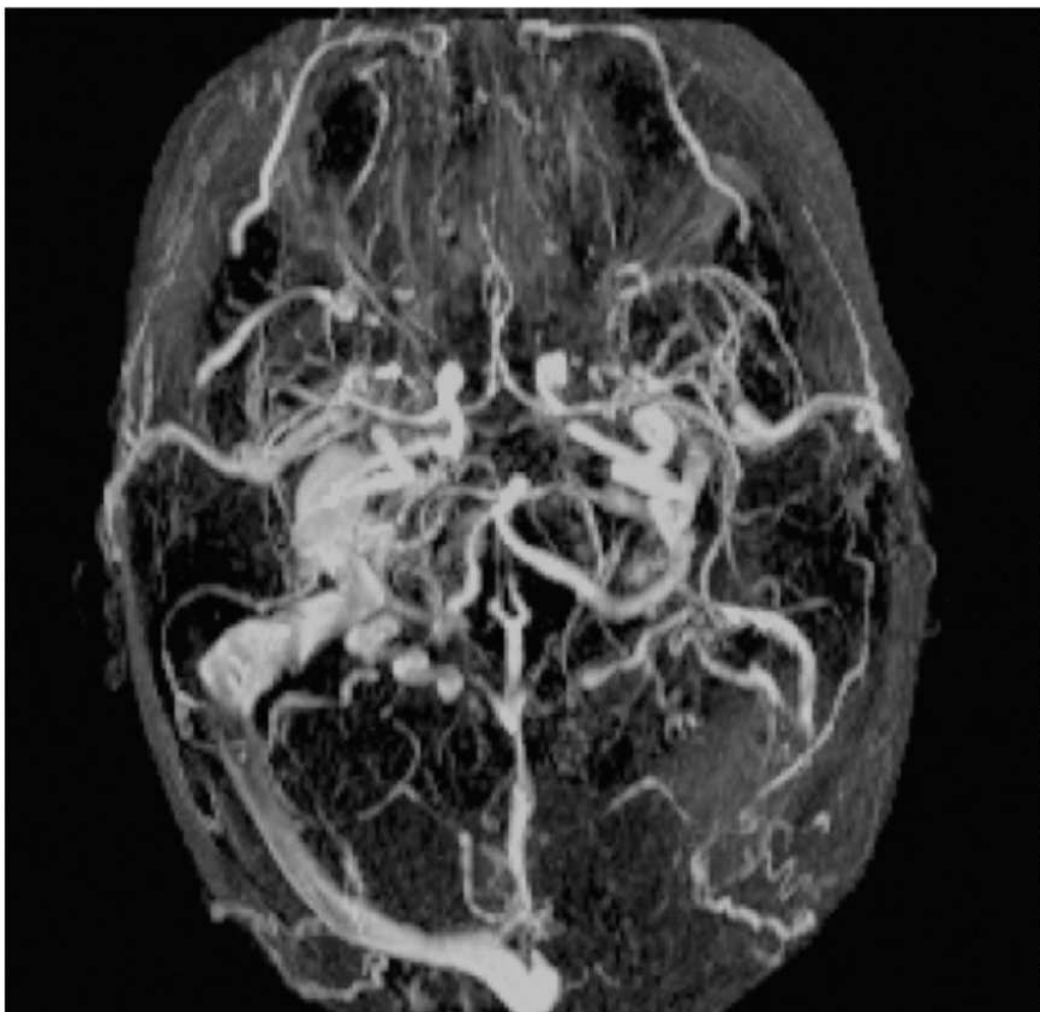


(c)

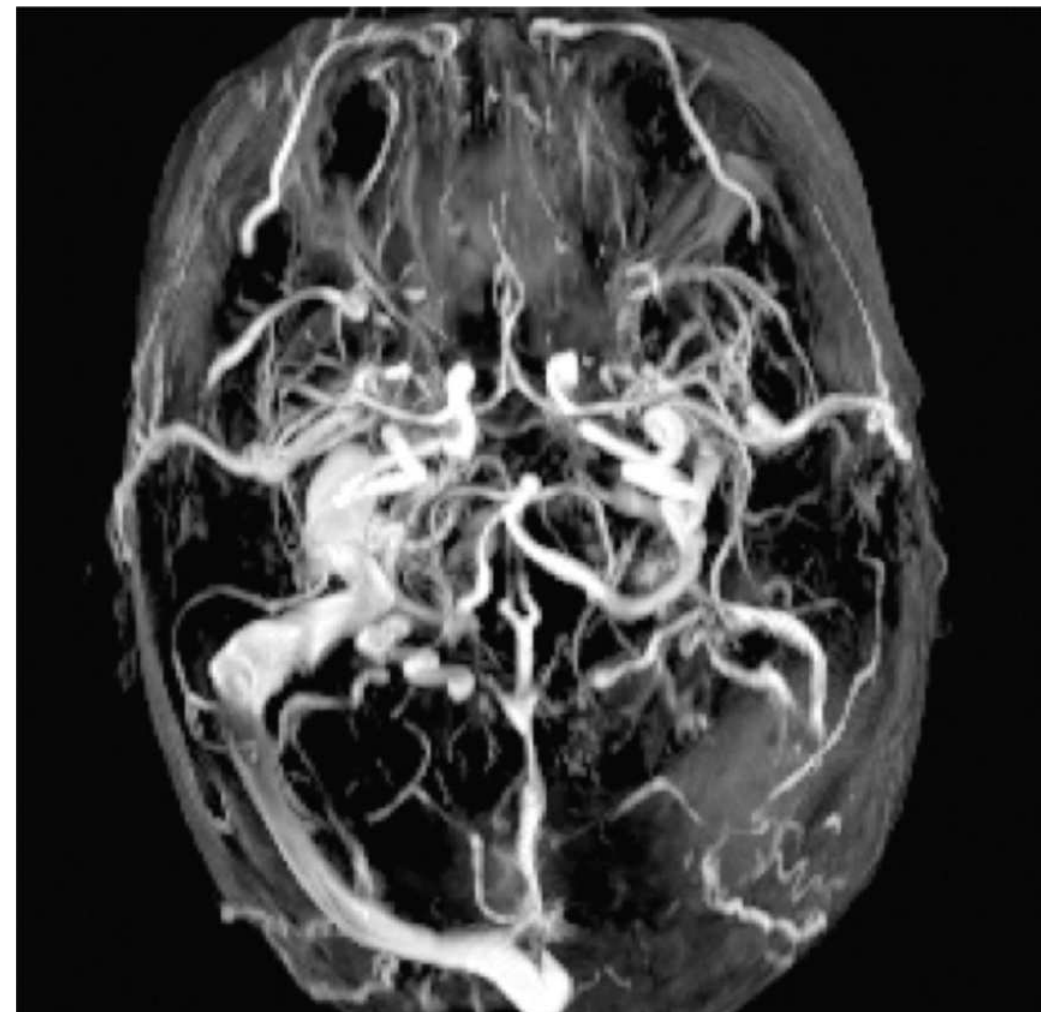
- a. Radiography of the skull, b. low-pass filter with a Gaussian filter (std=15, 20 x 20), c. high-pass Filter obtained from subtracting b from a.



# Adaptive Filtering: Example head MRA



MIP of MRA data before filtering

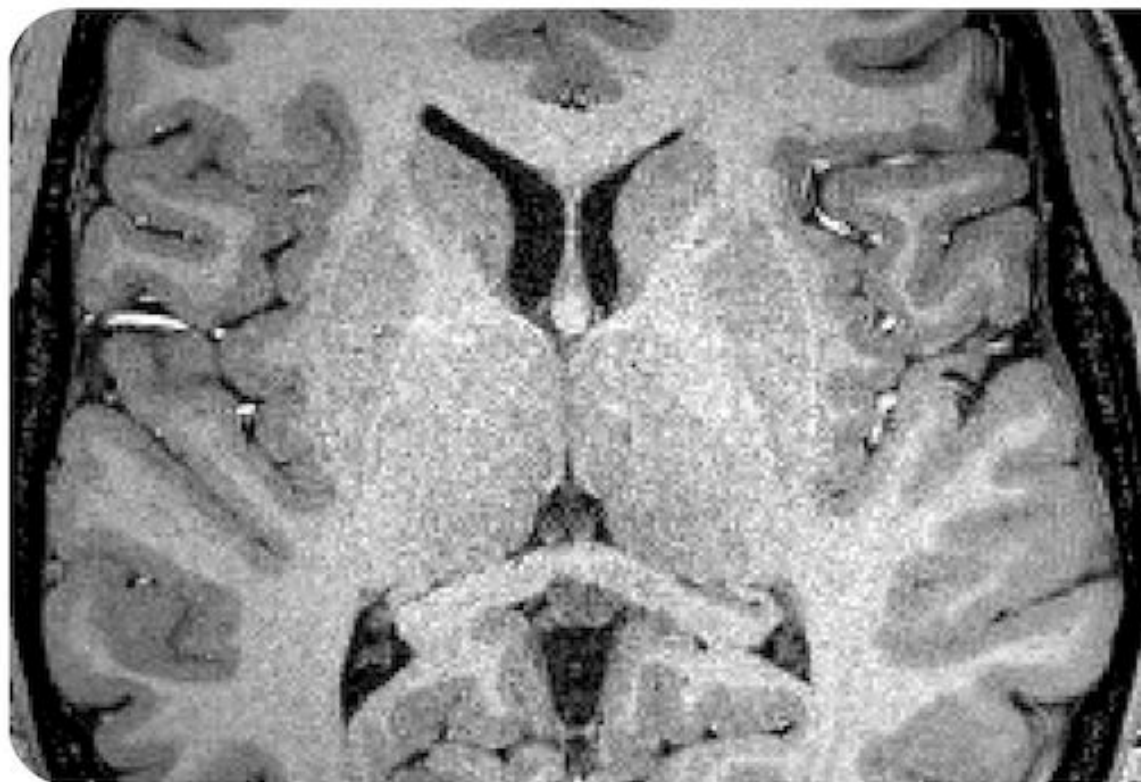


MIP of MRA data after filtering

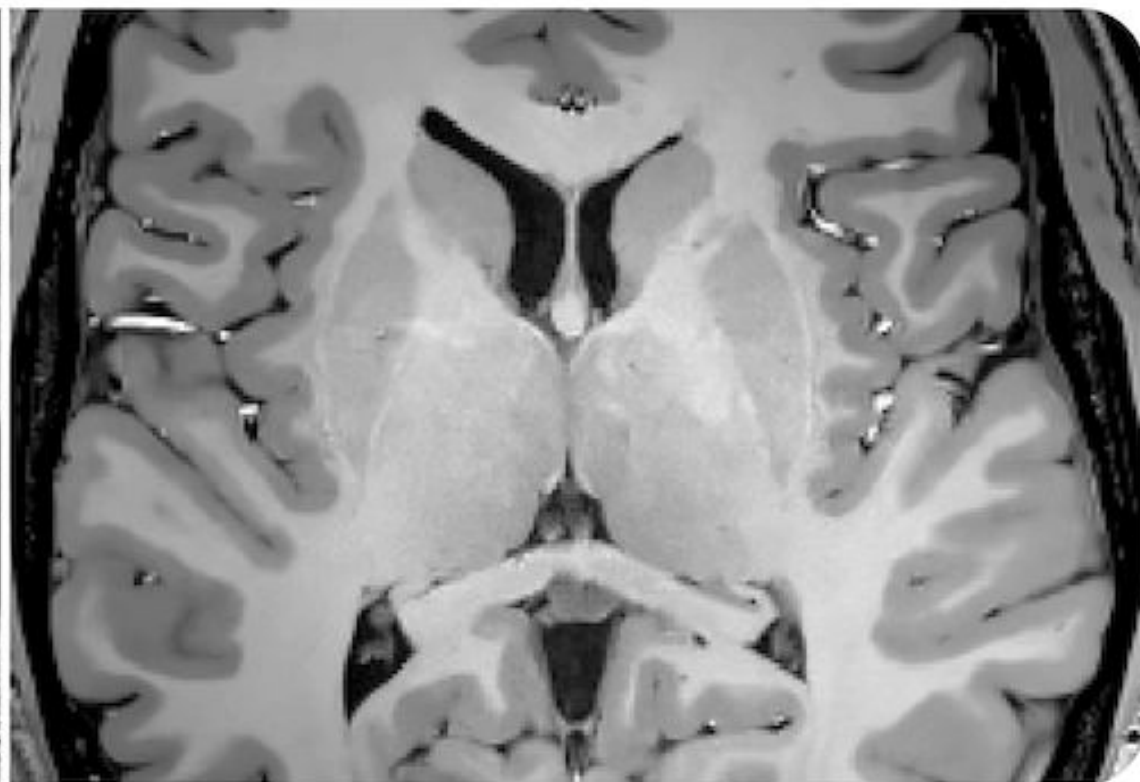


# Smoothing MRI (Denoising)

Before



After



© Dr Pierrick Coupé



# How to measure for evaluating noise removal algorithms?

**SNR:** Signal to Noise Ratio

**CNR:** Contrast to Noise Ratio

## MRI SNR – Example 1

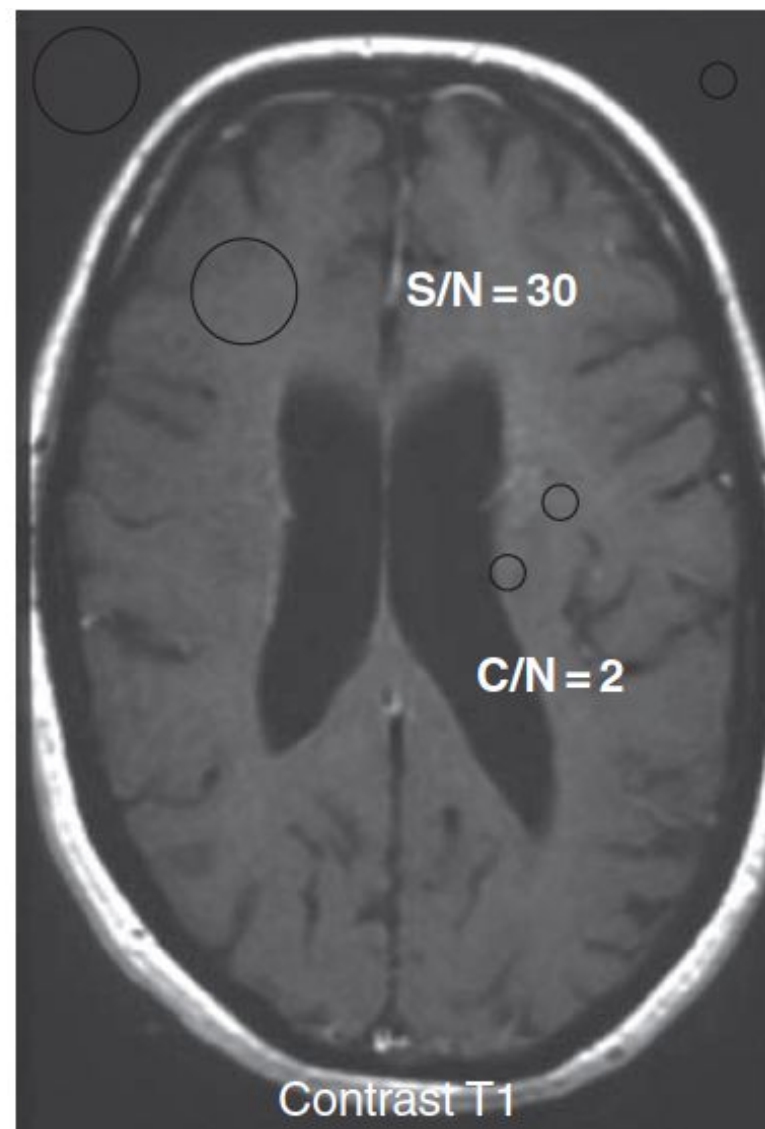
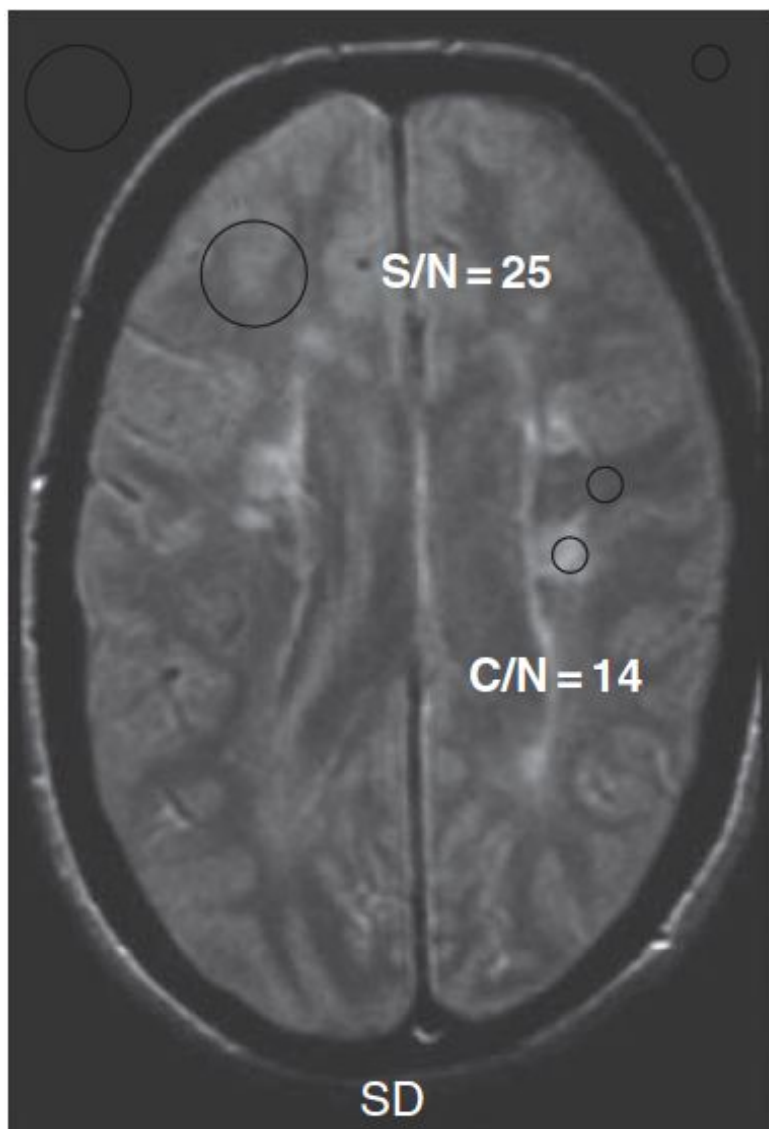
$$\text{SNR} = \frac{\text{Signal}}{\text{Noise}}$$

$$S = 700$$

$$N = 20$$

$$\begin{aligned}\text{SNR} &= 700 / 20 \\ &= 35\end{aligned}$$





SNR (S/N) (of normal brain) and CNR (C/N) of multiple sclerosis plaques to normal brain on spin-density and T<sub>1</sub> magnetic resonance images.

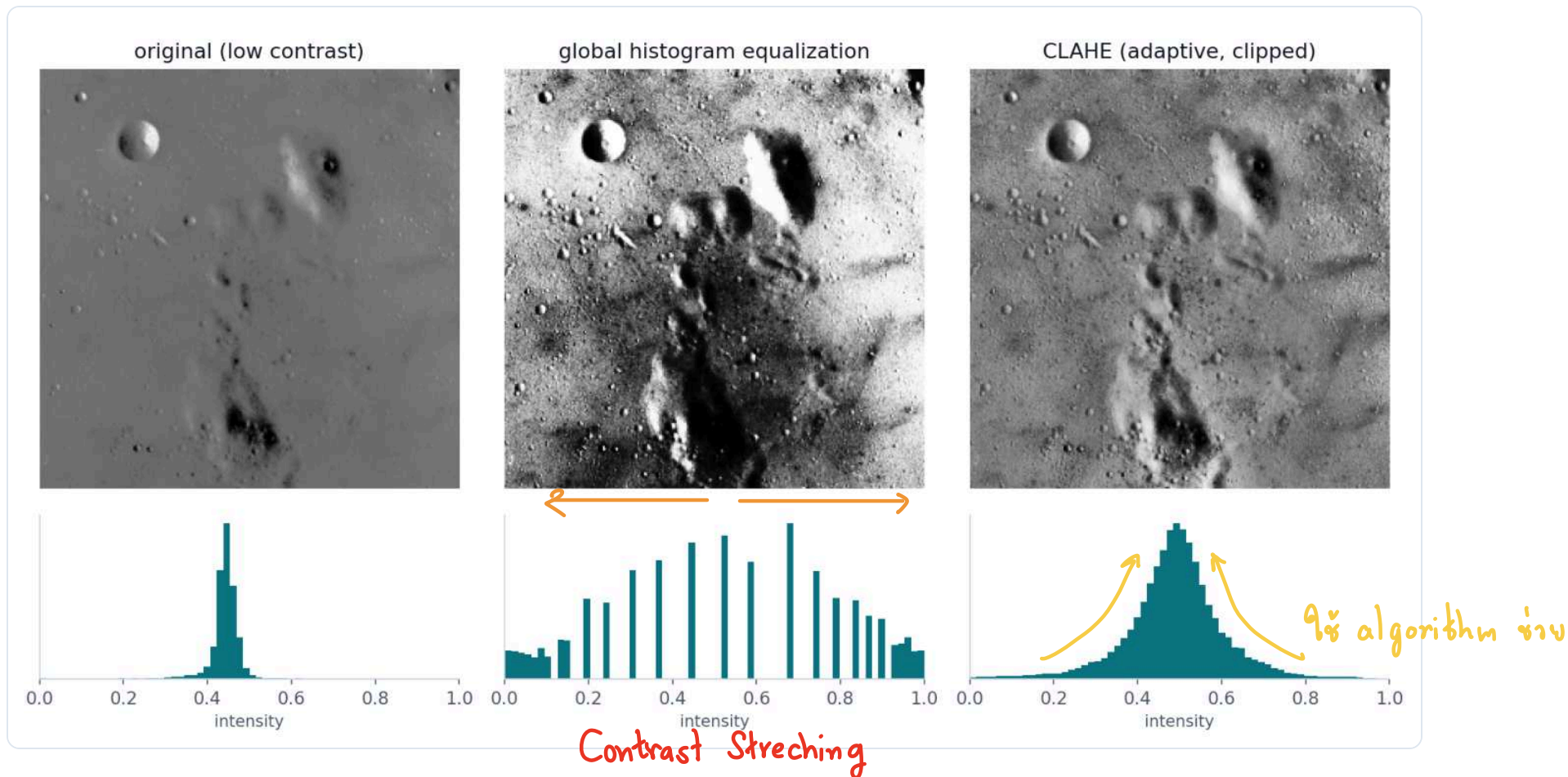
# Contrast, color, crop: the everyday trio

*Histogram equalization and CLAHE, color-space conversions, cropping and padding. Small operations, big footprint: they appear in nearly every published pipeline.*

AUTHORED

Written for this course; live examples computed with **scikit-image** and **OpenCV** on scikit-image sample images.  
[scikit-image.org](https://scikit-image.org) · [docs.opencv.org](https://docs.opencv.org)

# Histogram equalization, global and adaptive



Global equalization spreads the histogram but can wash regions out and amplify noise; CLAHE equalizes in tiles with a clip limit. CLAHE is the standard trick on chest X-rays and fundus images.

# Two lines each, one parameter that matters

```
# scikit-image
from skimage import exposure
eq    = exposure.equalize_hist(img)
clahe = exposure.equalize_adapthist(
    img, clip_limit=0.03)

# OpenCV (8-bit grayscale)
op  = cv2.createCLAHE(clipLimit=2.0,
                     tileGridSize=(8, 8))
out = op.apply(gray_u8)
```

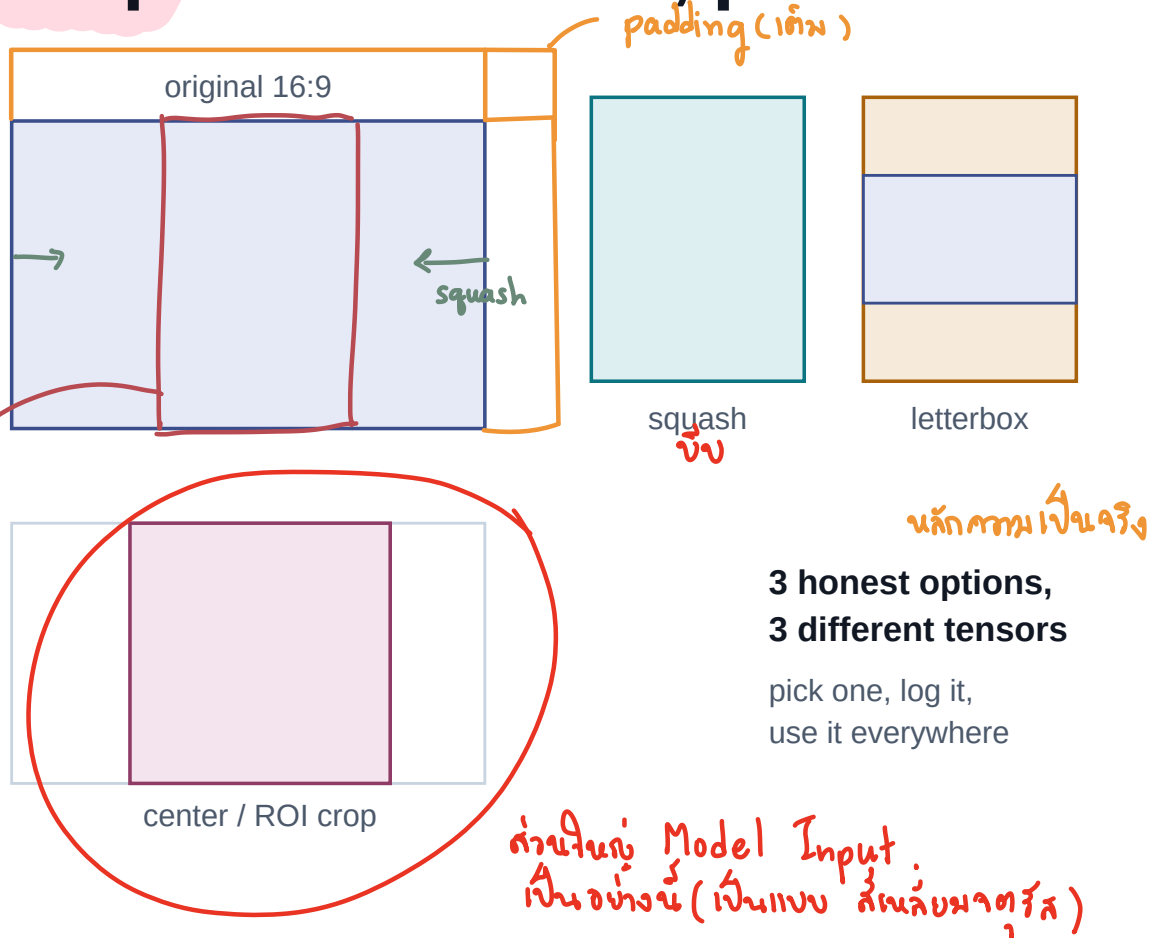
- **clip limit** controls how much contrast amplification is allowed per tile; too high and you equalize the noise.
- Report it. "CLAHE" without the clip limit and tile size is **not reproducible** (session 4 clinic, again).
- Decide **where it runs**: as offline preprocessing for every image, or as a probabilistic augmentation (Albumentations has `A.CLAHE(p=0.5)`). Those are different experiments.

# Color decisions live in the pipeline, not the model

- **Gray or RGB is an experiment variable.** Dropping to one channel cuts compute and can help small ultrasound datasets; it deletes information for dermatology and histopathology. Decide with a controlled run, not by habit.
- **Be consistent end to end:** the OpenCV BGR trap from class 1 returns the moment deployment code decodes with a different library than training did.
- **Domain shift is often a color shift.** Three scanners, three color casts (recall CMU's three-camera balloon slide). Histopathology fixes it with **stain normalization** in Lab-like spaces: Reinhard, Macenko, Vahadane (course topic 3.13).
- **Ultrasound tip from our lab's work:** strip the vendor overlay and colormap before the array reaches the model; the pixels must mean tissue, not GUI.

*If a human expert cannot say what an intensity means after your conversion, the model inherits the confusion.*

# Crop what matters, pad what remains



- **Region-of-interest cropping is domain knowledge as code:** our OPG-SHAP work cropped panoramic radiographs to the mandible region before classification (course topic 3.2). Big accuracy for free, zero new labels.
- **Padding value matters:** zeros create fake black borders the model can exploit; reflect or edge padding is usually safer. Log which one you used.
- **Random crops belong to augmentation** (part F); deterministic crops belong here, applied identically at train and test.

# Preprocessing for the model: why optimizers care

*Zero-centering, scaling, per-channel statistics. The gradient argument for why all-positive inputs slow learning, straight from the deck that modernized this material.*

SOURCE DECK

**Lecture 10, Training Neural Networks, Part 1**, EECS 498-007 / 598-005 Deep Learning for Computer Vision, Fall 2020 ·

Justin Johnson, University of Michigan.

<https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/> · © the author · shown unmodified · selected pages

# Lecture 10: Training Neural Networks (Part 1)

Consider what happens when nonlinearity is always positive

$$h_i^{(\ell)} = \sum_j w_{i,j}^{(\ell)} \sigma \left( h_j^{(\ell-1)} \right) + b_i^{(\ell)}$$

$h_i^{(\ell)}$  is the  $i$ th element of the hidden layer at layer  $\ell$  (before activation)

$w^{(\ell)}, b^{(\ell)}$  are the weights and bias of layer  $\ell$

What can we say about the gradients on  $w^{(\ell)}$ ?

Consider what happens when nonlinearity is always positive

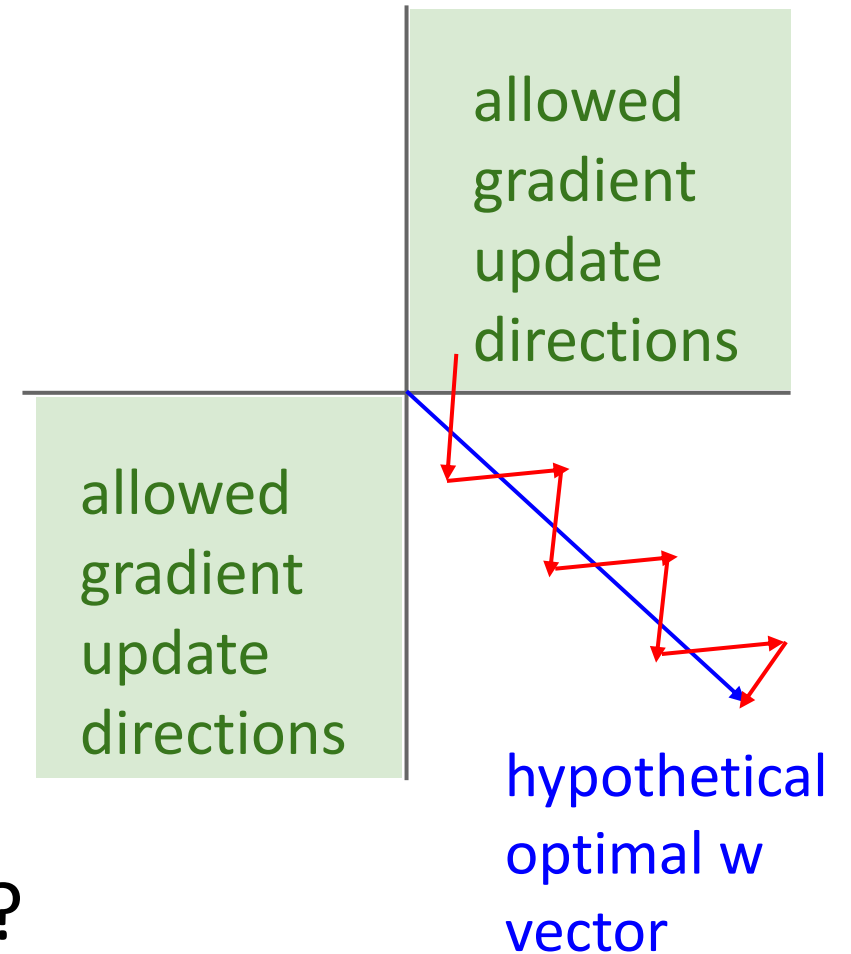
$$h_i^{(\ell)} = \sum_j w_{i,j}^{(\ell)} \sigma(h_j^{(\ell-1)}) + b_i^{(\ell)}$$

$h_i^{(\ell)}$  is the  $i$ th element of the hidden layer at layer  $\ell$  (before activation)

$w^{(\ell)}, b^{(\ell)}$  are the weights and bias of layer  $\ell$

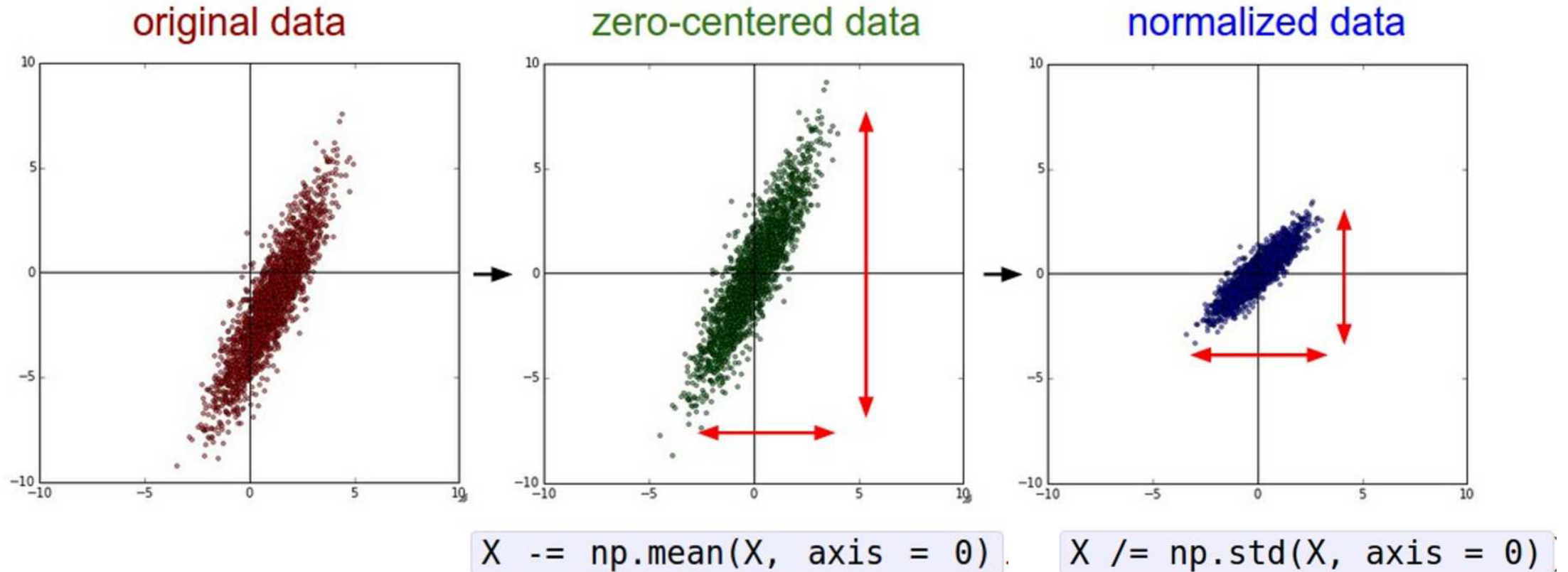
What can we say about the gradients on  $w^{(\ell)}$ ?

Always all positive or all negative :(



# Data Preprocessing

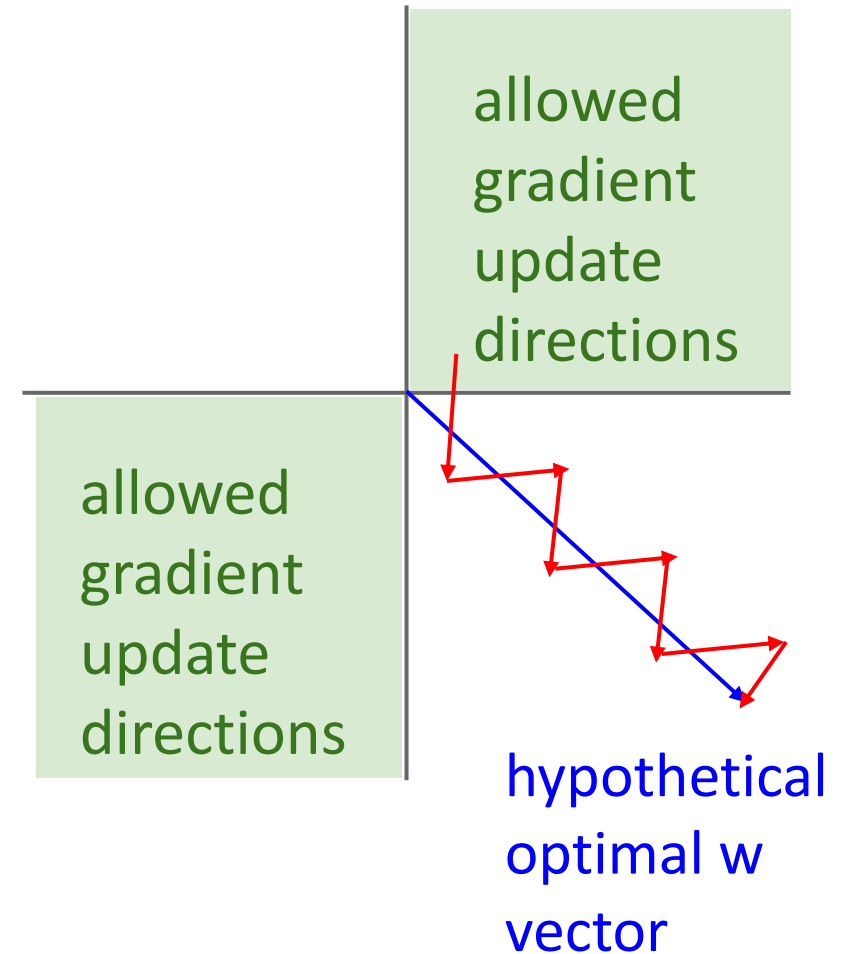
# Data Preprocessing



(Assume  $X$  [NxD] is data matrix,  
each example in a row)

Remember: Consider what happens when the input to a neuron is always positive...

$$h_i^{(\ell)} = \sum_j w_{i,j}^{(\ell)} \sigma(h_j^{(\ell-1)}) + b_i^{(\ell)}$$



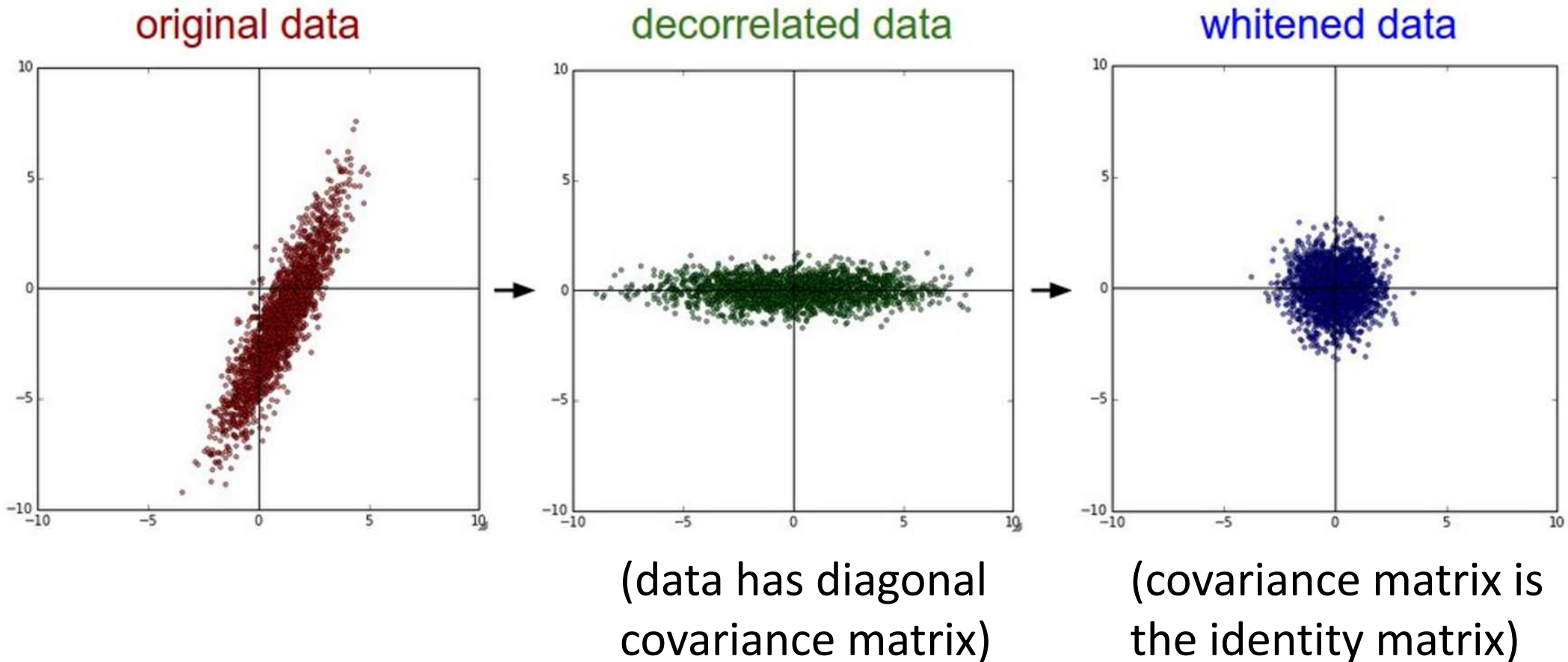
What can we say about the gradients on  $\mathbf{w}$ ?

Always all positive or all negative :(

(this is also why you want zero-mean data!)

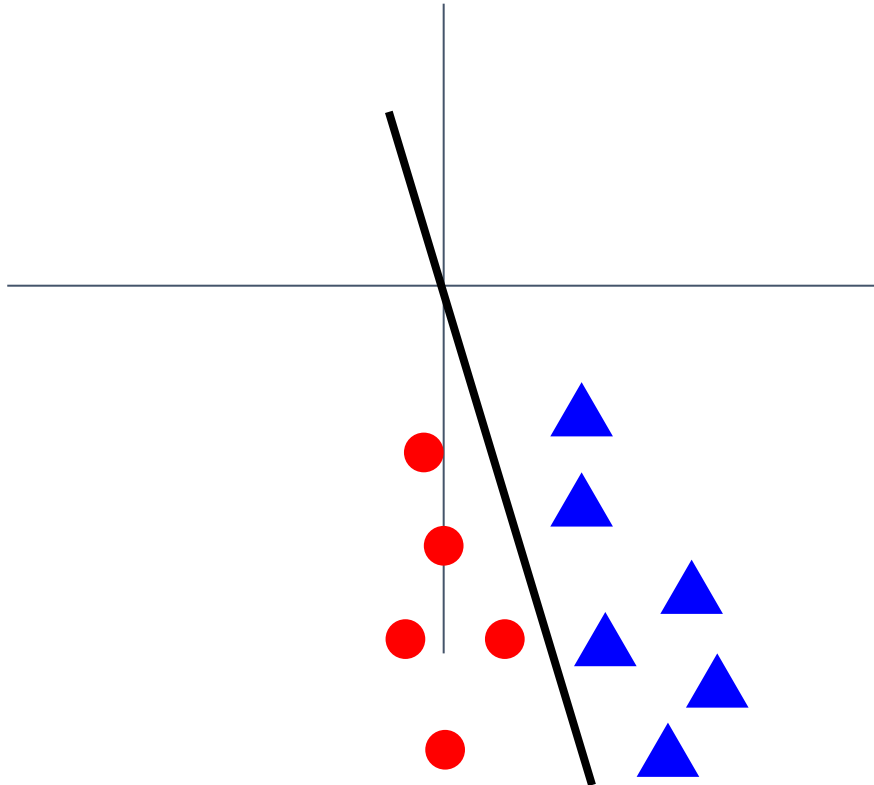
# Data Preprocessing

In practice, you may also see **PCA** and **Whitening** of the data

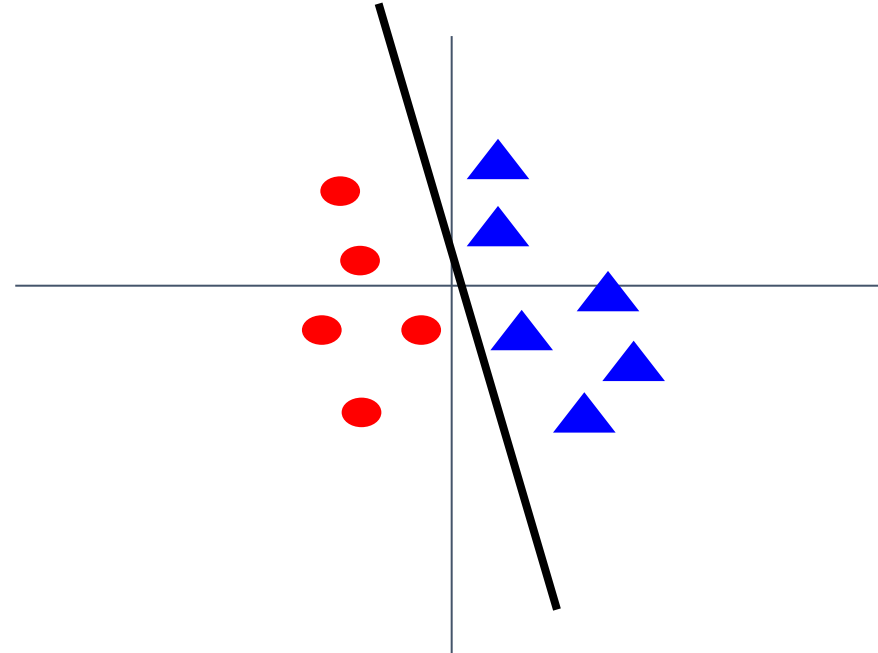


# Data Preprocessing

**Before normalization:** classification loss very sensitive to changes in weight matrix; hard to optimize



**After normalization:** less sensitive to small changes in weights; easier to optimize



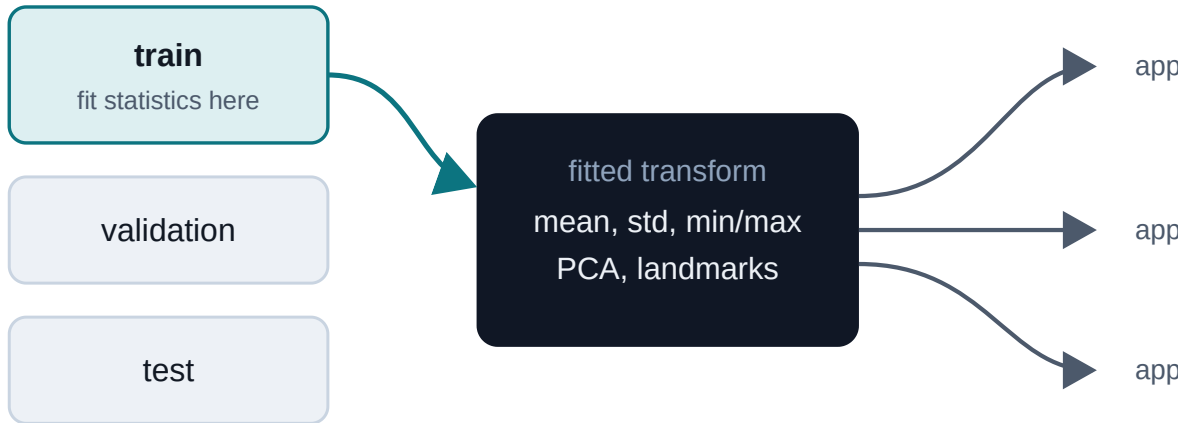
# Data Preprocessing for Images

e.g. consider CIFAR-10 example with [32,32,3] images

- Subtract the mean image (e.g. AlexNet)  
(mean image = [32,32,3] array)
- Subtract per-channel mean (e.g. VGGNet)  
(mean along each channel = 3 numbers)
- Subtract per-channel mean and  
Divide by per-channel std (e.g. ResNet)  
(mean along each channel = 3 numbers)

Not common to  
do PCA or  
whitening

# Fit preprocessing on the training split, only



**Fitting on all data = the test set leaks into training.**

Leakage taxonomy: Kapoor and Narayanan, Patterns 2023 (course section 1.11)

"Any preprocessing statistics (e.g. the data mean) must only be computed on the training data, and then applied to the validation / test data." (CS231n course notes, MIT license)

- Same rule at **patient level** for medical data: statistics and splits both respect the patient boundary.

# The leaky version and the honest version

```
# LEAKY: statistics see the test set
mean = images.mean()          # all data!
std  = images.std()
train, test = split(images)

# HONEST: split first, fit on train only
train, test = split(images)
mean = train.mean(axis=(0,1,2)) # per channel
std  = train.std(axis=(0,1,2))
train_n = (train - mean) / std
test_n  = (test  - mean) / std  # same numbers
```

- The honest version's test transform uses **frozen numbers**; nothing about test data ever influences them.
- With a small dataset the two versions differ by little until the day the dataset shifts, and then only the honest one tells the truth.
- **nnU-Net** automates exactly this: its dataset fingerprint (spacing, intensity percentiles, class ratios) is computed from training cases and stored (course topic 3.13).

# MRI intensity standardization: normalization with anatomy in it

*The same tissue lands on different intensities in every MR acquisition. Standardization learns a mapping from training images, then applies it everywhere: our rule, discovered by medical imaging decades ago.*

SOURCE DECK

**Image preprocessing**, BME 495 Deep Learning for Biomedical Imaging, Winter 2025, Lectures 3–4 · Ulas Bagci, Northwestern University.

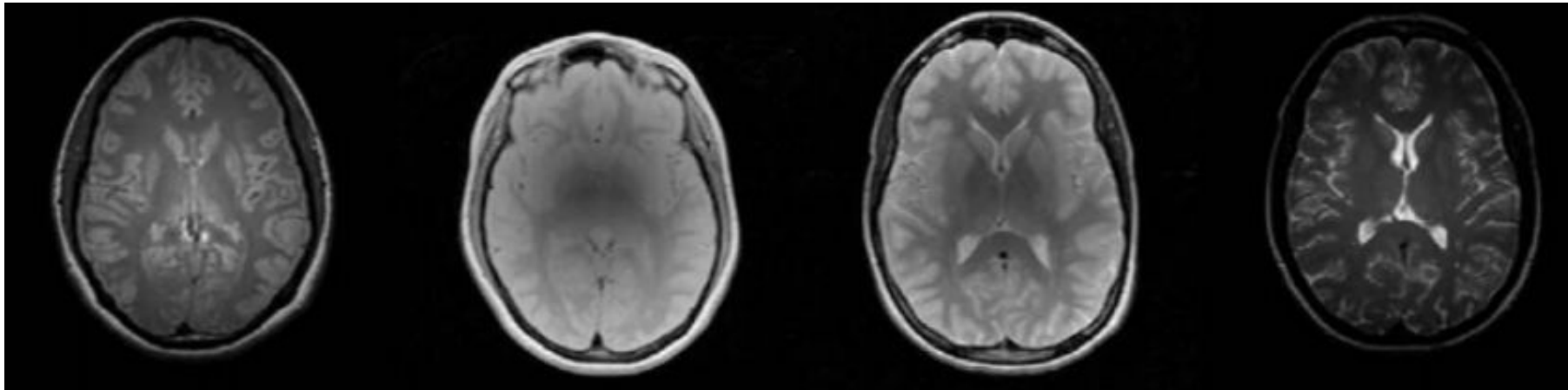
<https://bagcilab.com/courses/> publicly shared by the author · shown unmodified · selected pages (method: Nyul and Udupa)

# MRI Intensity Standardization

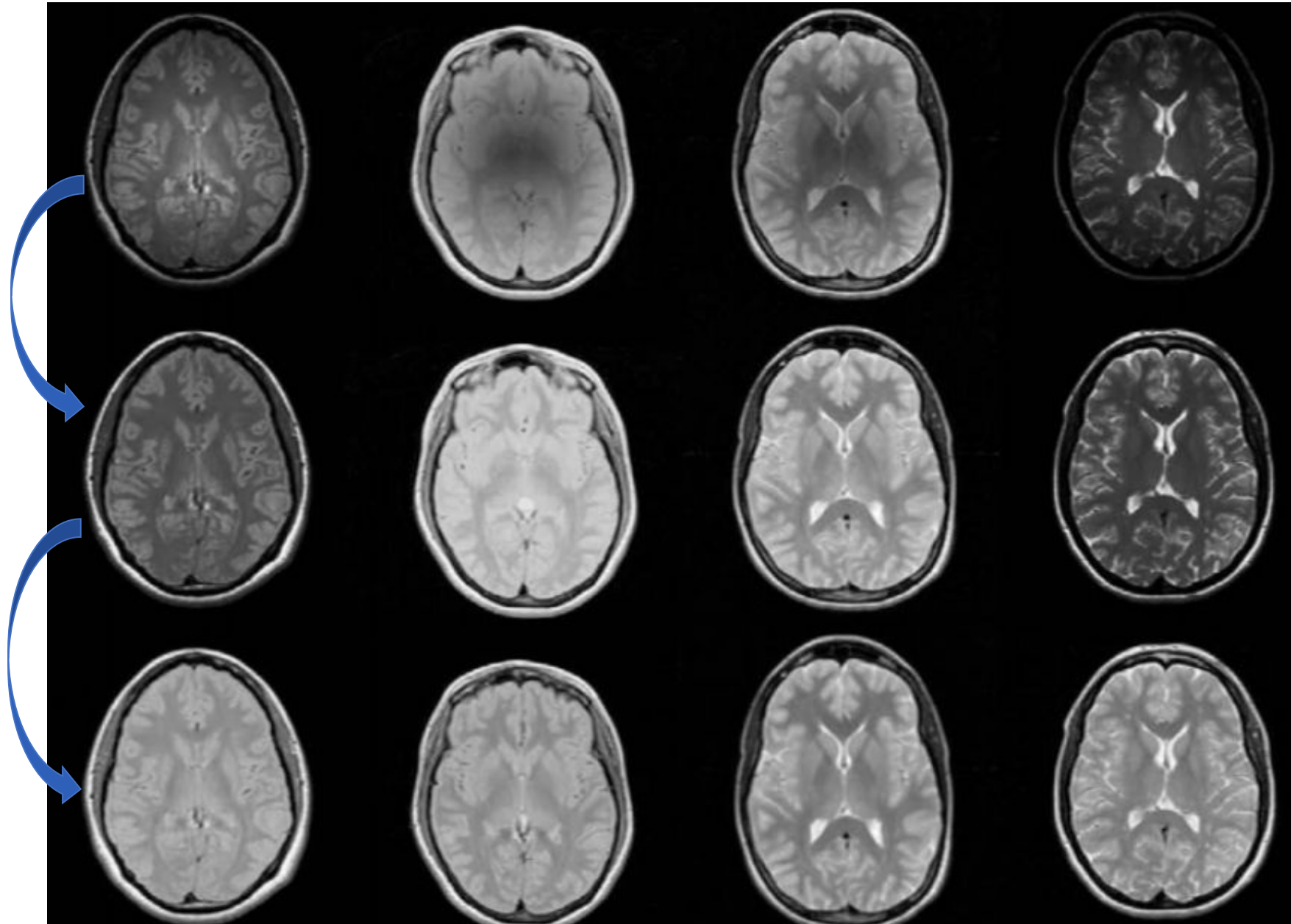


# MR Intensity Non-Standardness

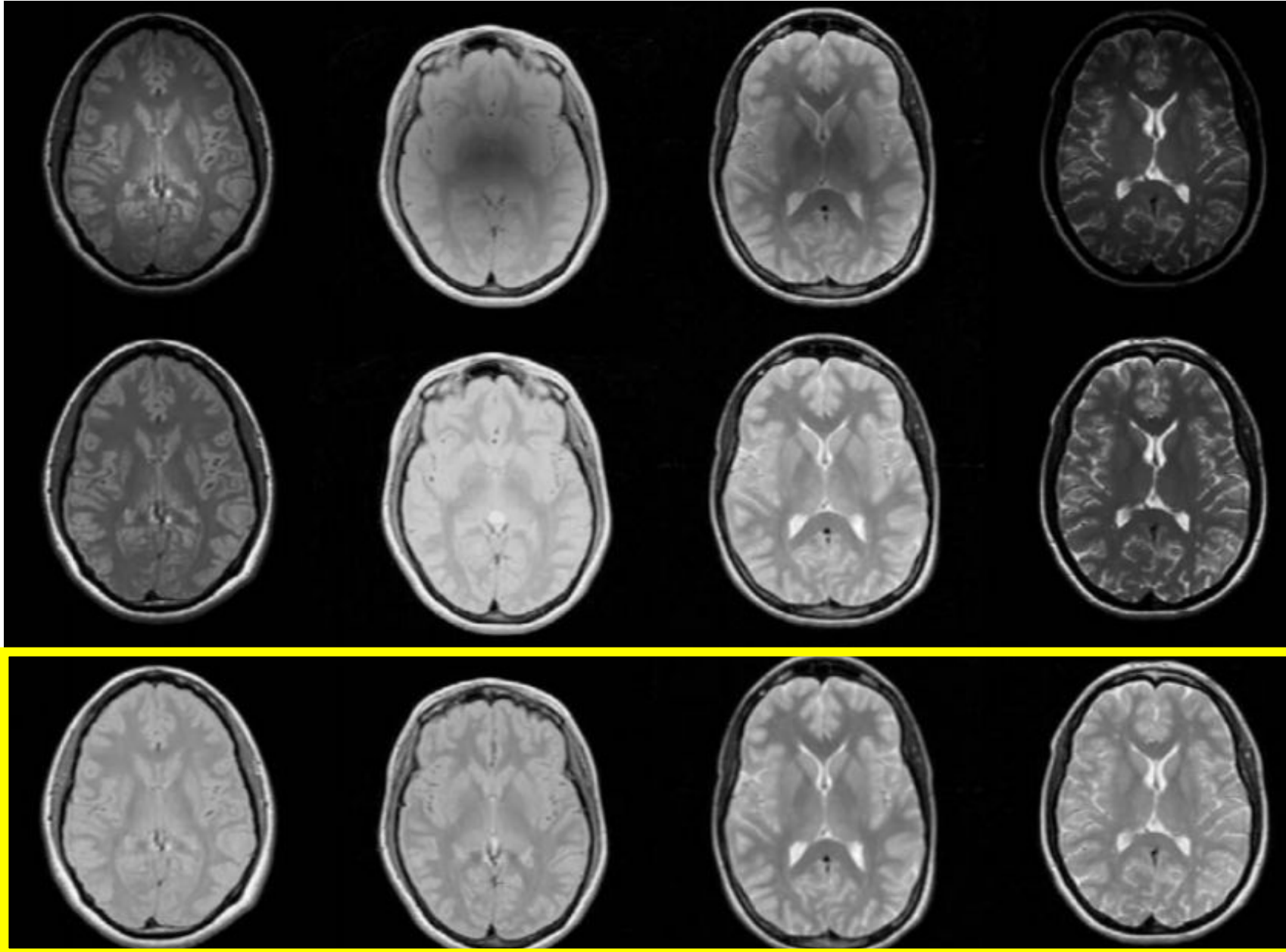
- Acquisition-to-acquisition signal intensity variations (non-standardness) are inherent in MR images.



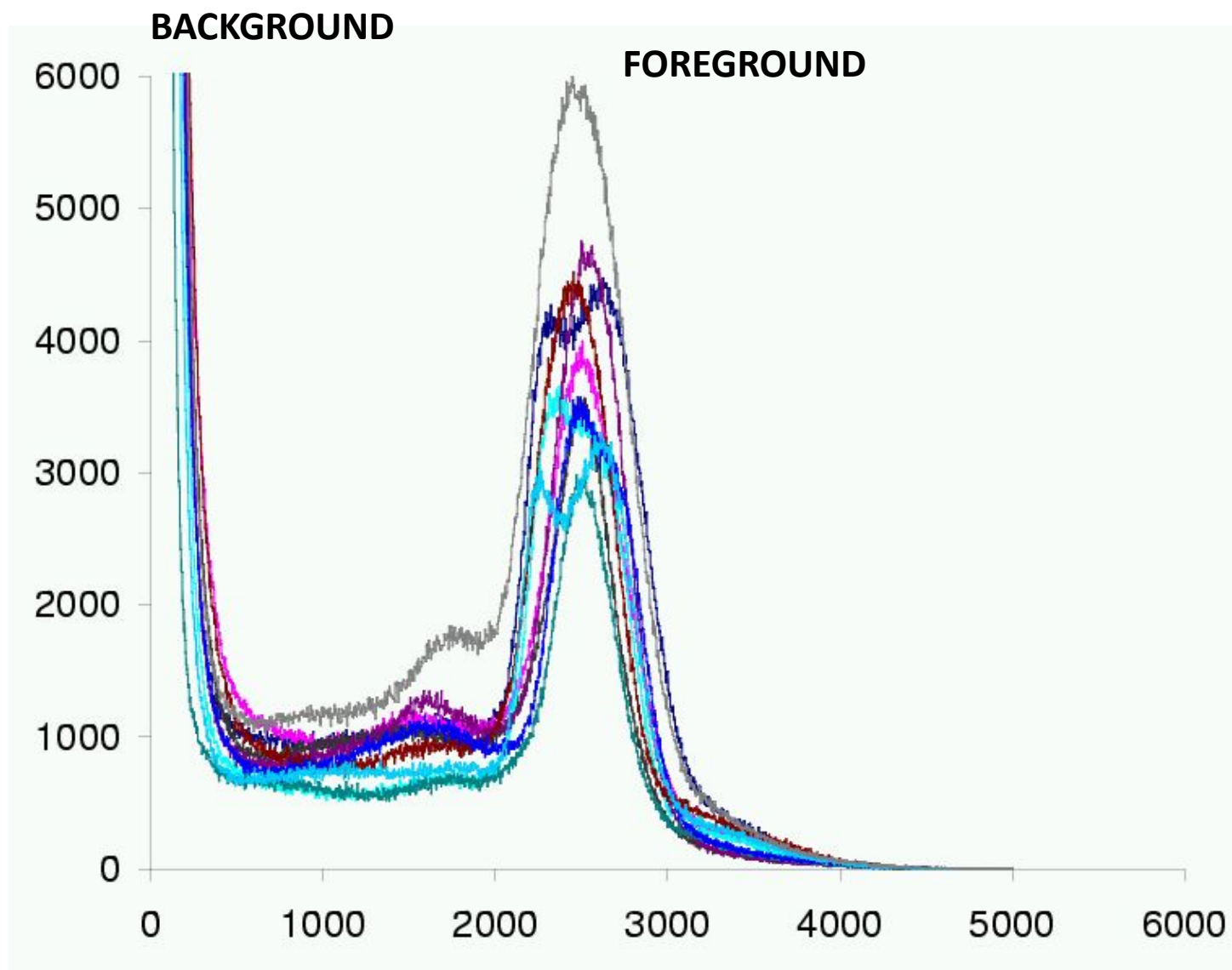
What is changed now?



Intensities are standardized!



**AFTER  
STANDARDIZATION**



10 Different  
PD studies

ORIGINAL  
HISTOGRAMS

**Credit: L. Nyul**



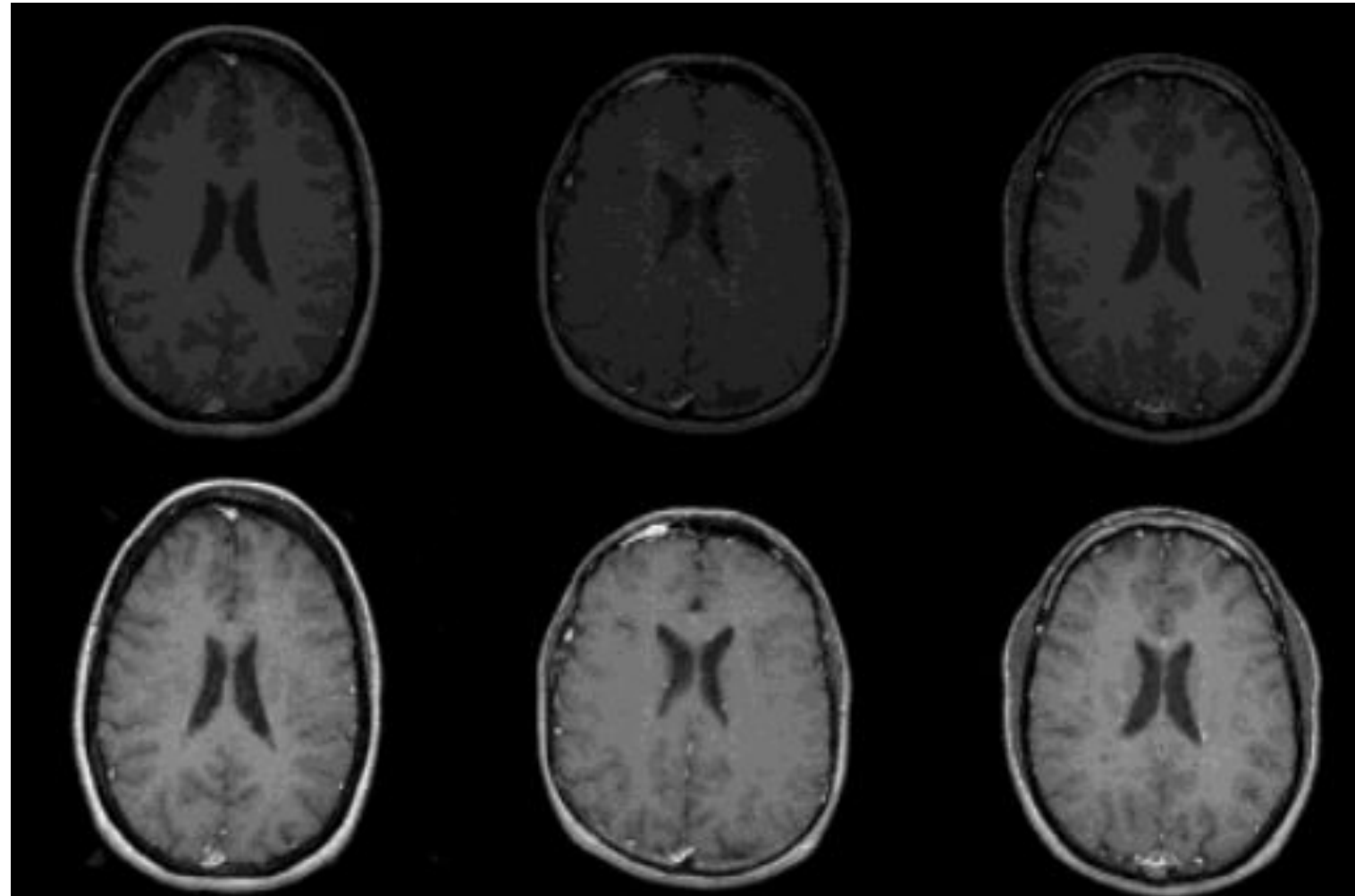
# Intensity Mapping Function

$$\tau_{\mathcal{D}_i}(X) \begin{cases} \left[ \mu_s + (X - \mu_i) \frac{s_1 - \mu_s}{p_{1i} - \mu_i} \right], & \text{if } m_{1i} \leq X \leq \mu_i, \\ \left[ \mu_s + (X - \mu_i) \frac{s_2 - \mu_s}{p_{2i} - \mu_i} \right], & \text{if } \mu_i \leq X \leq m_{2i}, \end{cases}$$



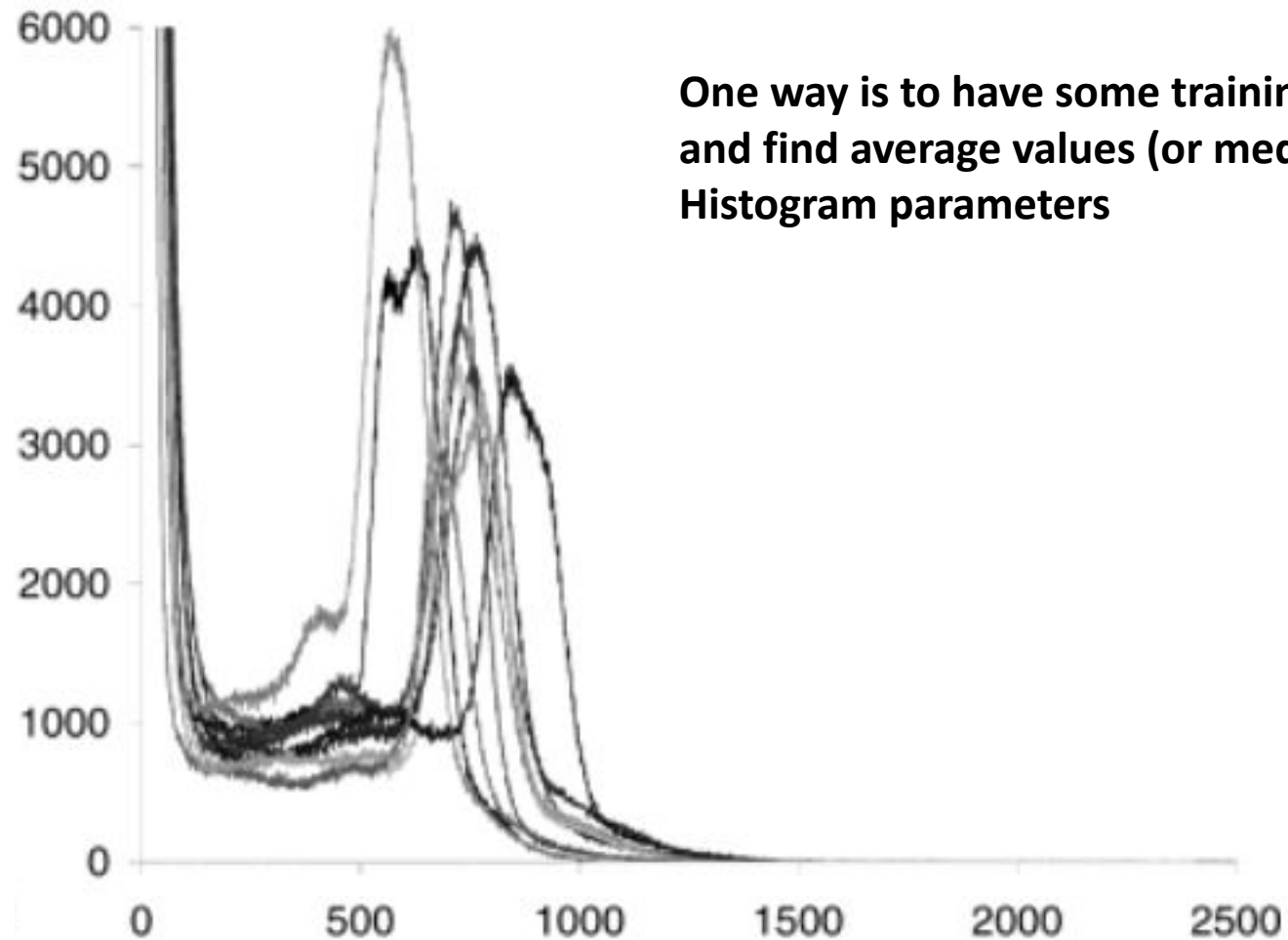
# Standardized Intensity Mapping T1-MRI

Original Scale



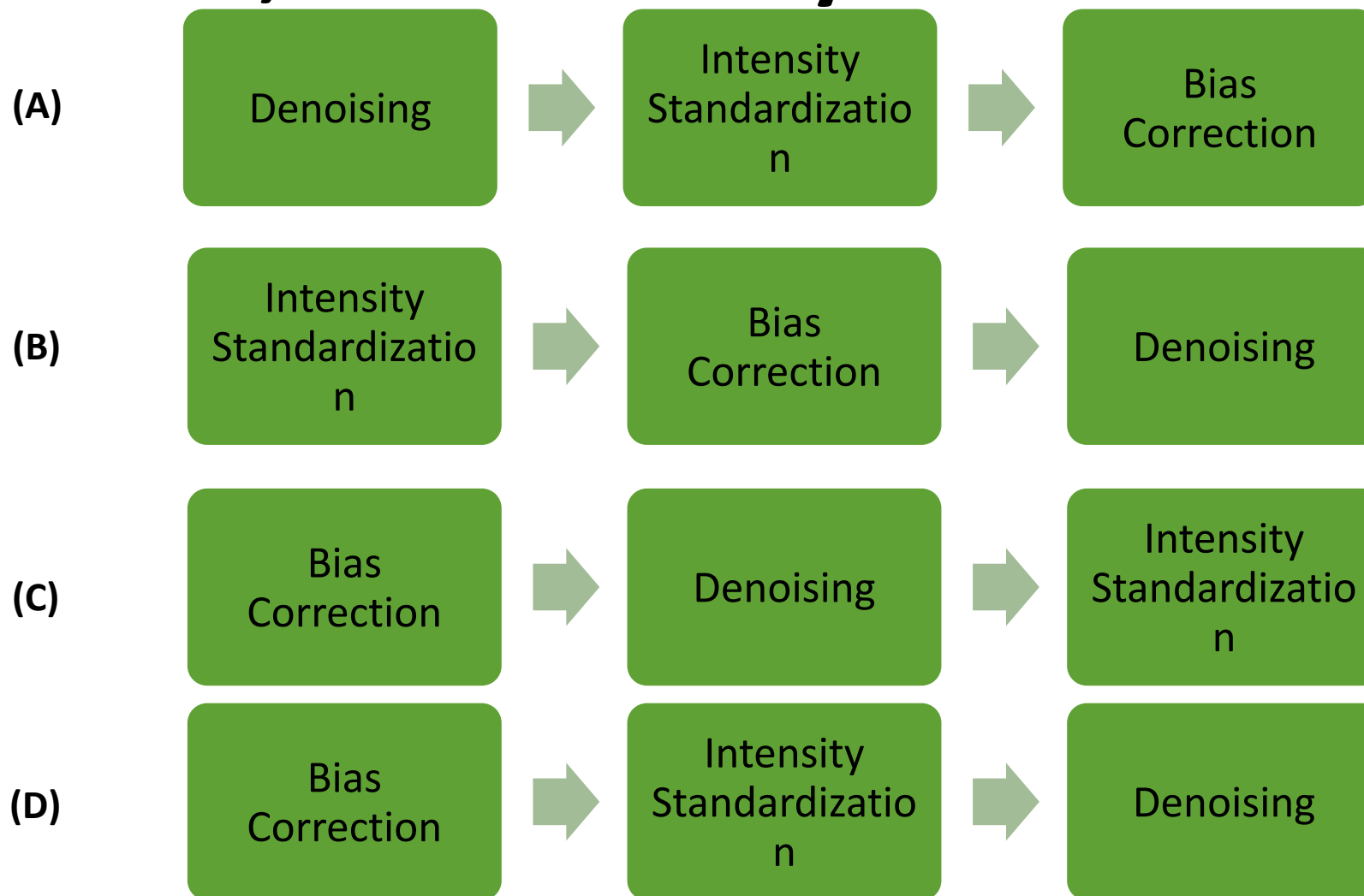
Standard Scale

# How to determine standardized parameters?



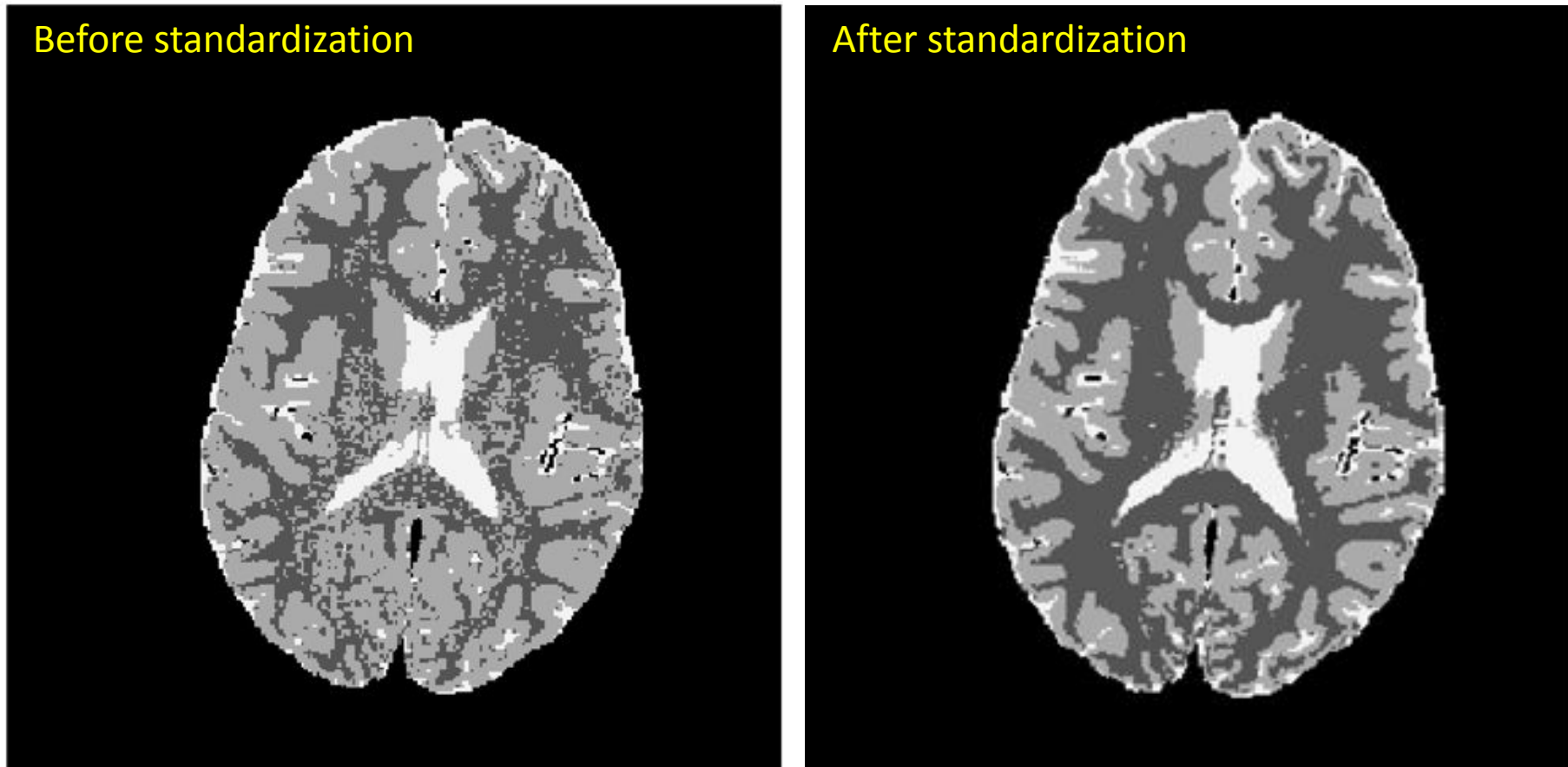
One way is to have some training images,  
and find average values (or median) for  
Histogram parameters

# Interplay between Denoising, Bias Correction, and Intensity Standardization ?



# Effects of Intensity Standardization on Image Segmentation

- Y.Zhuge et al (CVIU 2009) showed that intensity standardization **simplifies** brain image segmentation



# Summary

- ROI, VOI, SNR, CNR, Resolution, Noise, Thresholding, Histogram...are all important terminologies to know
- Intensity non-standardness is inherent in MRI
  - must standardize
- Intensity standardization + inhomogeneity correction + denoising need to be handled prior to image analysis
- Intensity non-standardness affects
  - Perception/qualitative analysis
  - Image analysis/quantification
    - Segmentation
    - Registration
    - Recognition/localization



# Transfer learning: borrowed weights, borrowed statistics

*Fine-tuning a pretrained backbone is the norm, and it quietly imports the preprocessing contract the backbone was trained with. Break the contract, lose the benefit.*

SOURCE DECK

**Lecture 7, Training Neural Networks, Part 2** (April 25, 2017), CS231n Convolutional Neural Networks for Visual Recognition

· Fei-Fei Li, Justin Johnson, Serena Yeung, Stanford University.

<https://cs231n.stanford.edu/> · © the authors · shown unmodified · selected pages · course notes are MIT-licensed at [cs231n.github.io](https://cs231n.github.io)

# Lecture 7: Training Neural Networks, Part 2

# Last time: Batch Normalization

**Input:**  $x : N \times D$

**Learnable params:**

$$\gamma, \beta : D$$

**Intermediates:**  $\mu, \sigma : D$   
 $\hat{x} : N \times D$

**Output:**  $y : N \times D$

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

# Transfer Learning

“You need a lot of a data if you want to train/use CNNs”

# Transfer Learning

“You need a lot of data if you want to train/use CNNs”

**BUSTED**



More specific

More generic

|                            |                             |                               |
|----------------------------|-----------------------------|-------------------------------|
|                            | <b>very similar dataset</b> | <b>very different dataset</b> |
| <b>very little data</b>    | ?                           | ?                             |
| <b>quite a lot of data</b> | ?                           | ?                             |



More specific

More generic

|                            |                                    |                               |
|----------------------------|------------------------------------|-------------------------------|
|                            | <b>very similar dataset</b>        | <b>very different dataset</b> |
| <b>very little data</b>    | Use Linear Classifier on top layer | ?                             |
| <b>quite a lot of data</b> | Finetune a few layers              | ?                             |



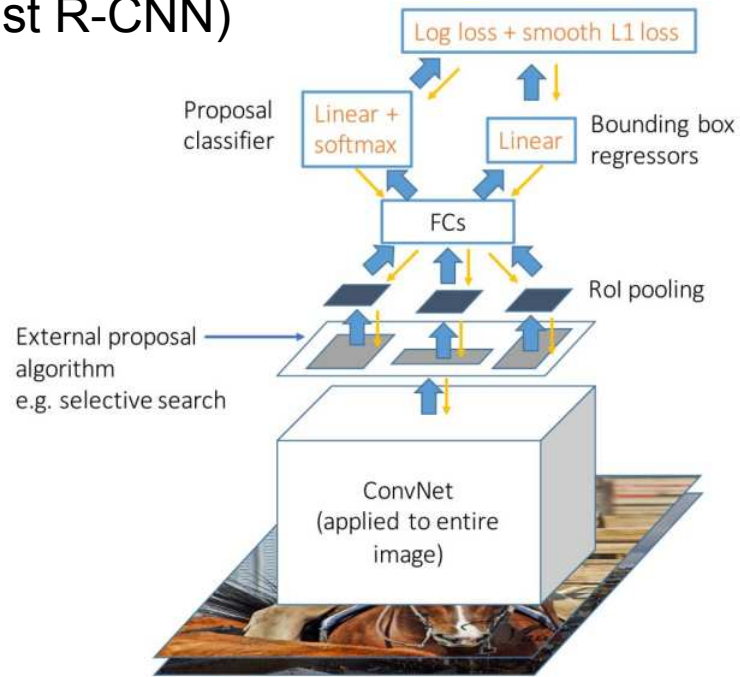
More specific

More generic

|                            |                                    |                                                                         |
|----------------------------|------------------------------------|-------------------------------------------------------------------------|
|                            | <b>very similar dataset</b>        | <b>very different dataset</b>                                           |
| <b>very little data</b>    | Use Linear Classifier on top layer | <u>You're in trouble...</u> Try linear classifier from different stages |
| <b>quite a lot of data</b> | <u>Finetune</u> a few layers       | Finetune a larger number of layers                                      |

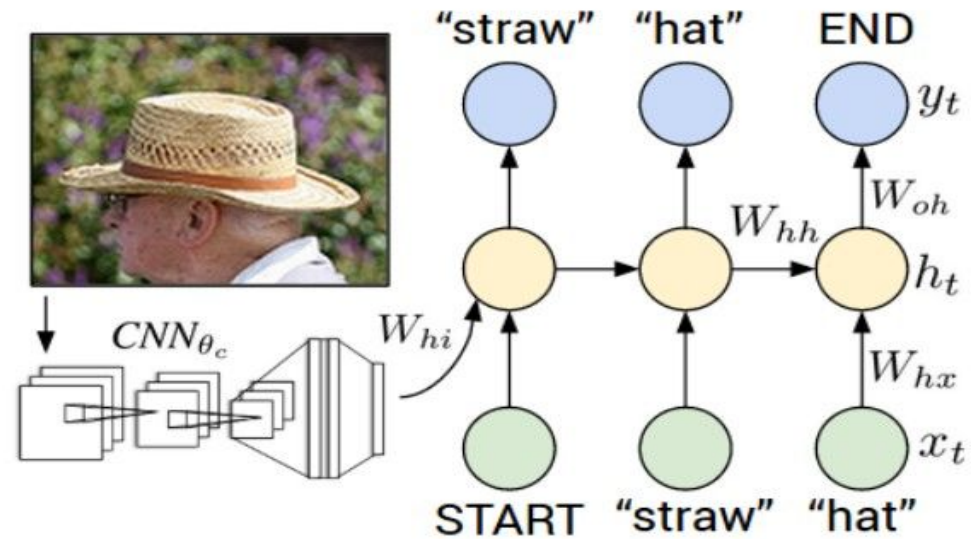
# Transfer learning with CNNs is pervasive... (it's the norm, not an exception)

## Object Detection (Fast R-CNN)



Girshick, "Fast R-CNN", ICCV 2015  
Figure copyright Ross Girshick, 2015. Reproduced with permission.

## Image Captioning: CNN + RNN



Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for  
Generating Image Descriptions", CVPR 2015  
Figure copyright IEEE, 2015. Reproduced for educational purposes.

# Takeaway for your projects and beyond:

Have some dataset of interest but it has  $< \sim 1\text{M}$  images?

1. Find a very large dataset that has similar data, train a big ConvNet there
2. Transfer learn to your dataset

Deep learning frameworks provide a “Model Zoo” of pretrained models so you don’t need to train your own

Caffe: <https://github.com/BVLC/caffe/wiki/Model-Zoo>

TensorFlow: <https://github.com/tensorflow/models>

PyTorch: <https://github.com/pytorch/vision>

# Those six numbers you keep copy-pasting

```
# The ImageNet statistics behind most backbones
mean = [0.485, 0.456, 0.406]
std  = [0.229, 0.224, 0.225]

tfm = v2.Compose([
    v2.Resize(256, antialias=True),
    v2.CenterCrop(224),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(mean=mean, std=std),
])
```

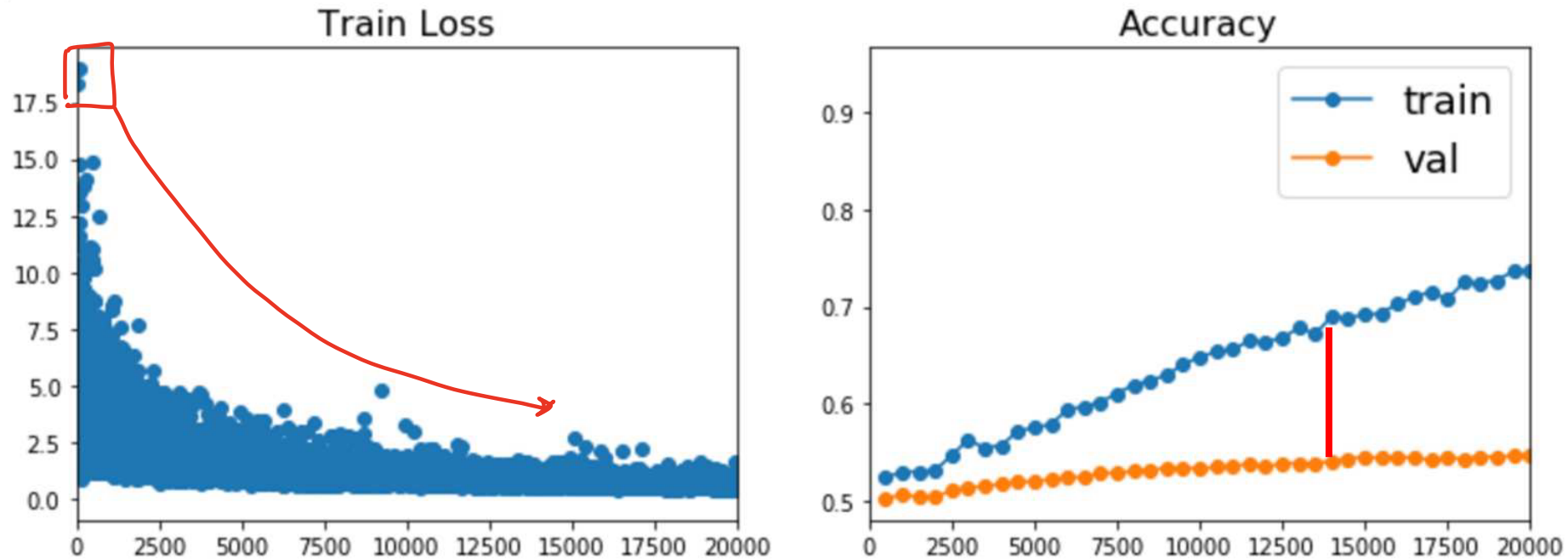
torchvision v2; identical idea in every framework.

- They are the **training-set statistics of ImageNet**: the rule from E2, applied by someone else, frozen into the checkpoint's contract.
- **Match the backbone**: many ViT and CLIP checkpoints use different constants (0.5-based, or CLIP's own). Read the model card, do not assume.
- Grayscale medical images through an RGB backbone: replicate to 3 channels **then** use the backbone's stats, or retrain the stem; document which.
- Wrong stats will not crash. They just shift every input off the distribution the network expects.

# Augmentation: preprocessing that changes every epoch

*Train-time randomness, test-time averaging. Flips, crops, color jitter, Cutout, MixUp: one pattern, many instances, and a knob for how much dataset you can fake.*

Now your model is training ... but it overfits!



Regularization

# Regularization: A common pattern

**Training:** Add some kind of randomness

$$y = f_W(x, z)$$

**Testing:** Average out randomness (sometimes approximate)

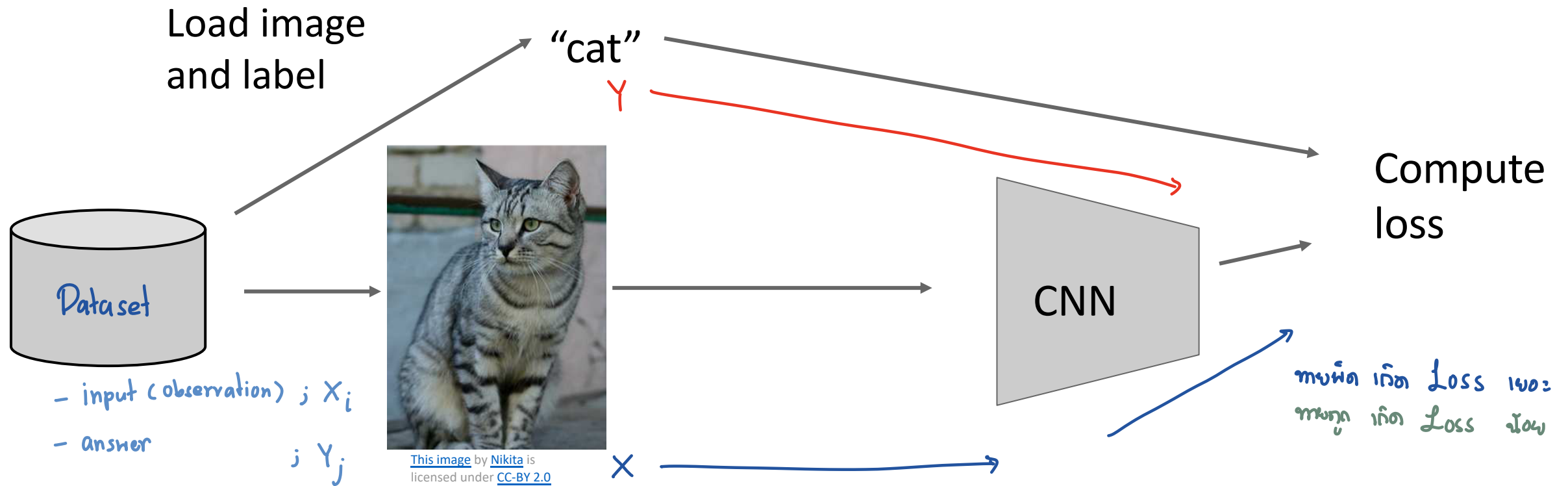
$$y = f(x) = E_z[f(x, z)] = \int p(z)f(x, z)dz$$

**Example:** Batch Normalization

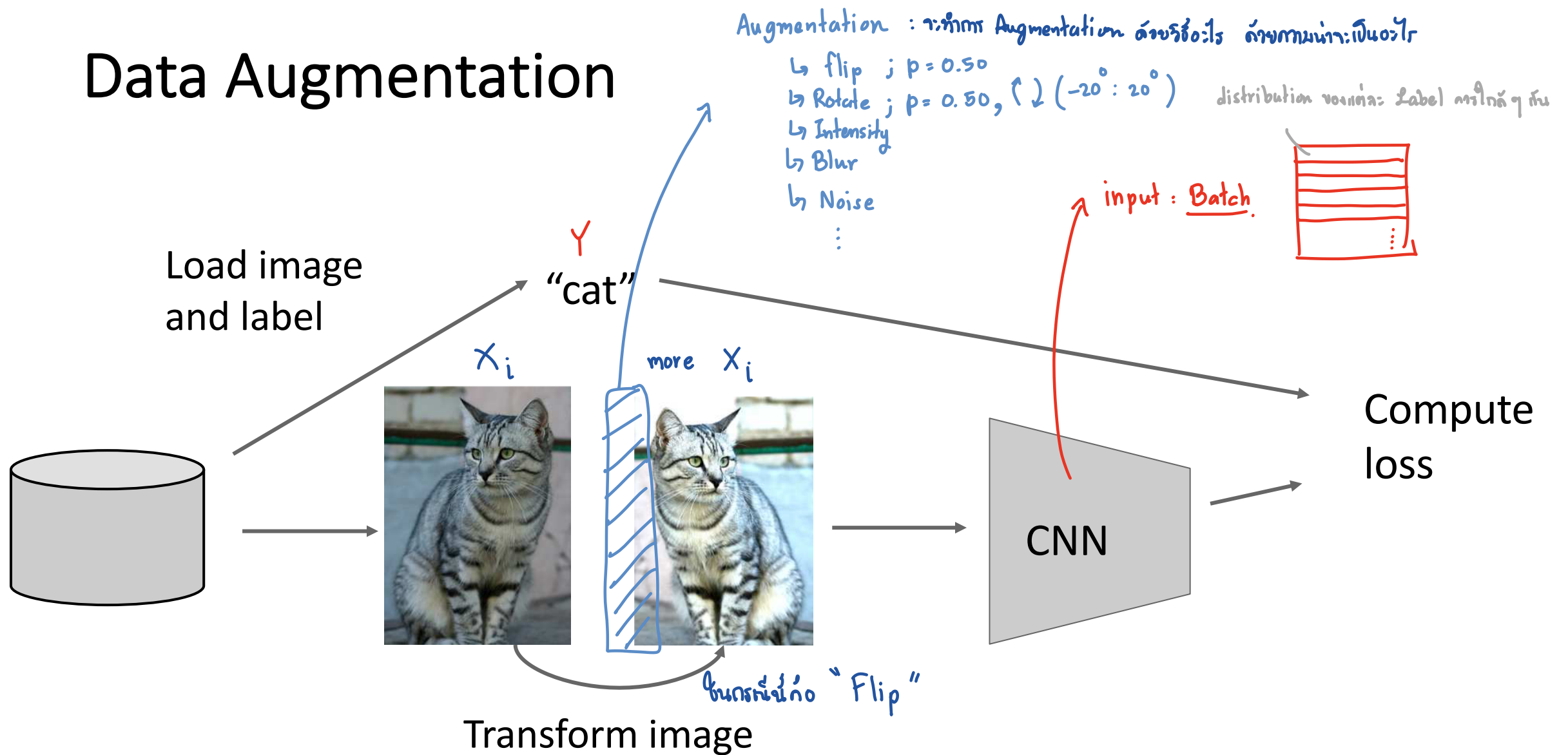
**Training:** Normalize using stats from random minibatches

**Testing:** Use fixed stats to normalize

# Data Augmentation



# Data Augmentation



# Data Augmentation: Horizontal Flips



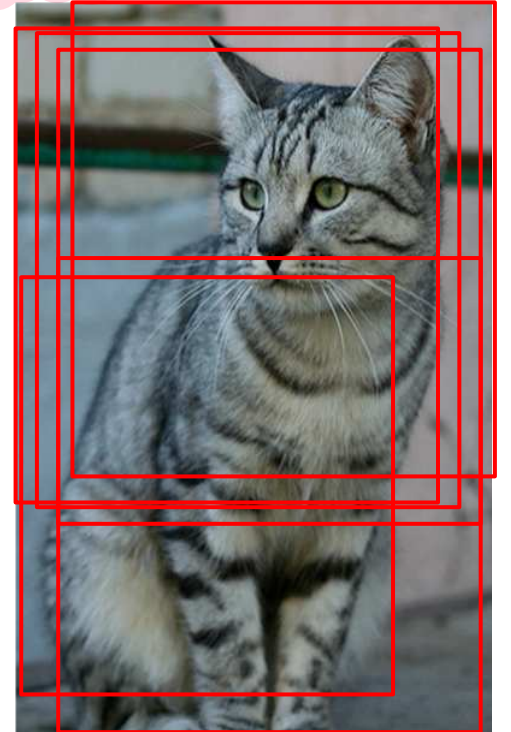
Vertical  
Flip

# Data Augmentation: Random Crops and Scales

**Training:** sample random crops / scales

ResNet:

1. Pick random  $L$  in range  $[256, 480]$
2. Resize training image, short side =  $L$
3. Sample random  $224 \times 224$  patch

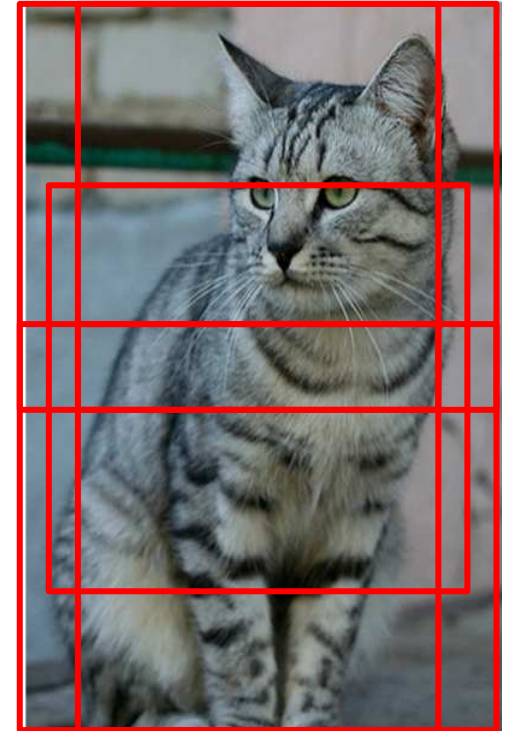


# Data Augmentation: Random Crops and Scales

**Training:** sample random crops / scales

ResNet:

1. Pick random  $L$  in range  $[256, 480]$
2. Resize training image, short side =  $L$
3. Sample random  $224 \times 224$  patch



**Testing:** average a fixed set of crops

ResNet:

1. Resize image at 5 scales:  $\{224, 256, 384, 480, 640\}$
2. For each size, use 10  $224 \times 224$  crops: 4 corners + center, + flips

# Data Augmentation: Color Jitter

Simple: Randomize  
contrast and brightness



## More Complex:

1. Apply PCA to all [R, G, B] pixels in training set
2. Sample a “color offset” along principal component directions
3. Add offset to all pixels of a training image

(Used in AlexNet, ResNet, etc)

# Data Augmentation: Get creative for your problem!

Random mix/combinations of :

- translation
- rotation
- stretching
- shearing,
- lens distortions, ... (go crazy)

# Regularization: A common pattern

**Training:** Add some randomness

**Testing:** Marginalize over randomness

## **Examples:**

Dropout

Batch Normalization

Data Augmentation

# Regularization: Stochastic Depth

**Training:** Set random images regions to 0

**Testing:** Use the whole image

## Examples:

Dropout

Batch Normalization

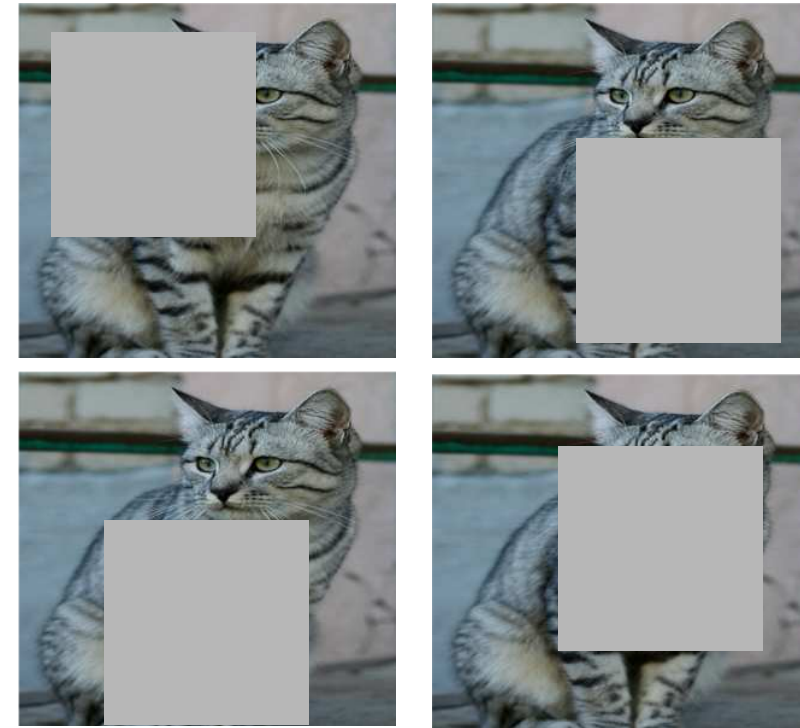
Data Augmentation

DropConnect

Fractional Max Pooling

Stochastic Depth

Cutout



Works very well for small datasets like CIFAR, less common for large datasets like ImageNet

# Regularization: Mixup

**Training:** Train on random blends of images

**Testing:** Use original images

## Examples:

Dropout

Batch Normalization

Data Augmentation

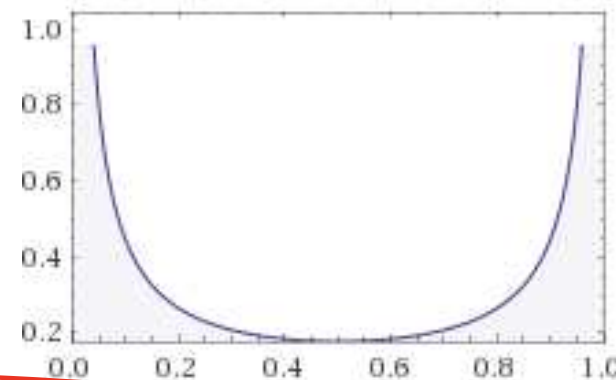
DropConnect

Fractional Max Pooling

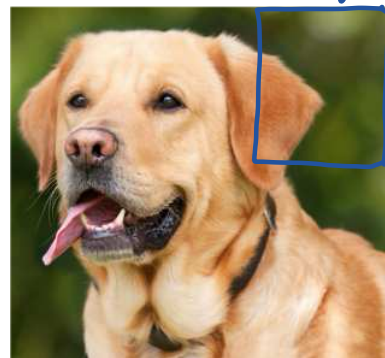
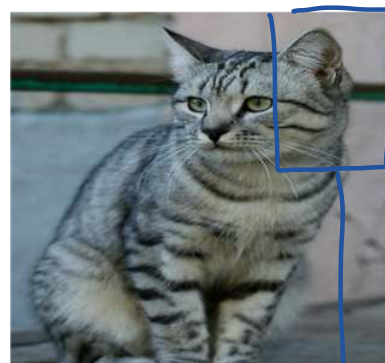
Stochastic Depth

Cutout

Mixup



Sample blend probability from a beta distribution  $\text{Beta}(a, b)$  with  $a=b \approx 0$  so blend weights are close to 0/1



Randomly blend the pixels of pairs of training images, e.g. 40% cat, 60% dog

CNN

Target label:  
cat: 0.4  
dog: 0.6

ทำอย่างนี้  
model ก็หาไม่ได้

Zhang et al, "mixup: Beyond Empirical Risk Minimization", ICLR 2018

∴ ส่วน Cutmix ด้วย

# Regularization: Mixup

**Training:** Train on random blends of images

**Testing:** Use original images

## Examples:

Dropout

Batch Normalization

Data Augmentation

DropConnect

Fractional Max Pooling

Stochastic Depth

Cutout

Mixup

- Consider dropout for large fully-connected layers
- Batch normalization and data augmentation almost always a good idea
- Try cutout and mixup especially for small classification datasets

Zhang et al, “*mixup*: Beyond Empirical Risk Minimization”, ICLR 2018

# Summary

## **1. One time setup**

Activation functions, data preprocessing, weight initialization, regularization

**Today**

## **2. Training dynamics**

Learning rate schedules; large-batch training; hyperparameter optimization

**Next time**

## **3. After training**

Model ensembles, transfer learning

# Medical augmentation: fake data, real physics

*Ghosting, spike noise, motion, bias fields: medical augmentation can simulate the scanner itself, not just geometry. Plus the honest limits: where MixUp should not go.*

SOURCE DECK

**Lecture 9, Evaluating DL Algorithms and Data Augmentation**, BME 495, Winter 2025 · Ulas Bagci, Northwestern University.

<https://bagcilab.com/courses/> publicly shared by the author · shown unmodified · selected pages (review: Chlap et al. 2021)

# Data Augmentation

a technique used in machine learning and deep learning to increase the diversity of data available for training models, without actually collecting new data.

- **Common Techniques:** Rotation, flipping, cropping, noise addition, affine transformations.
- **Advanced Techniques:** Generative models (e.g., GANs), MixUp, etc.

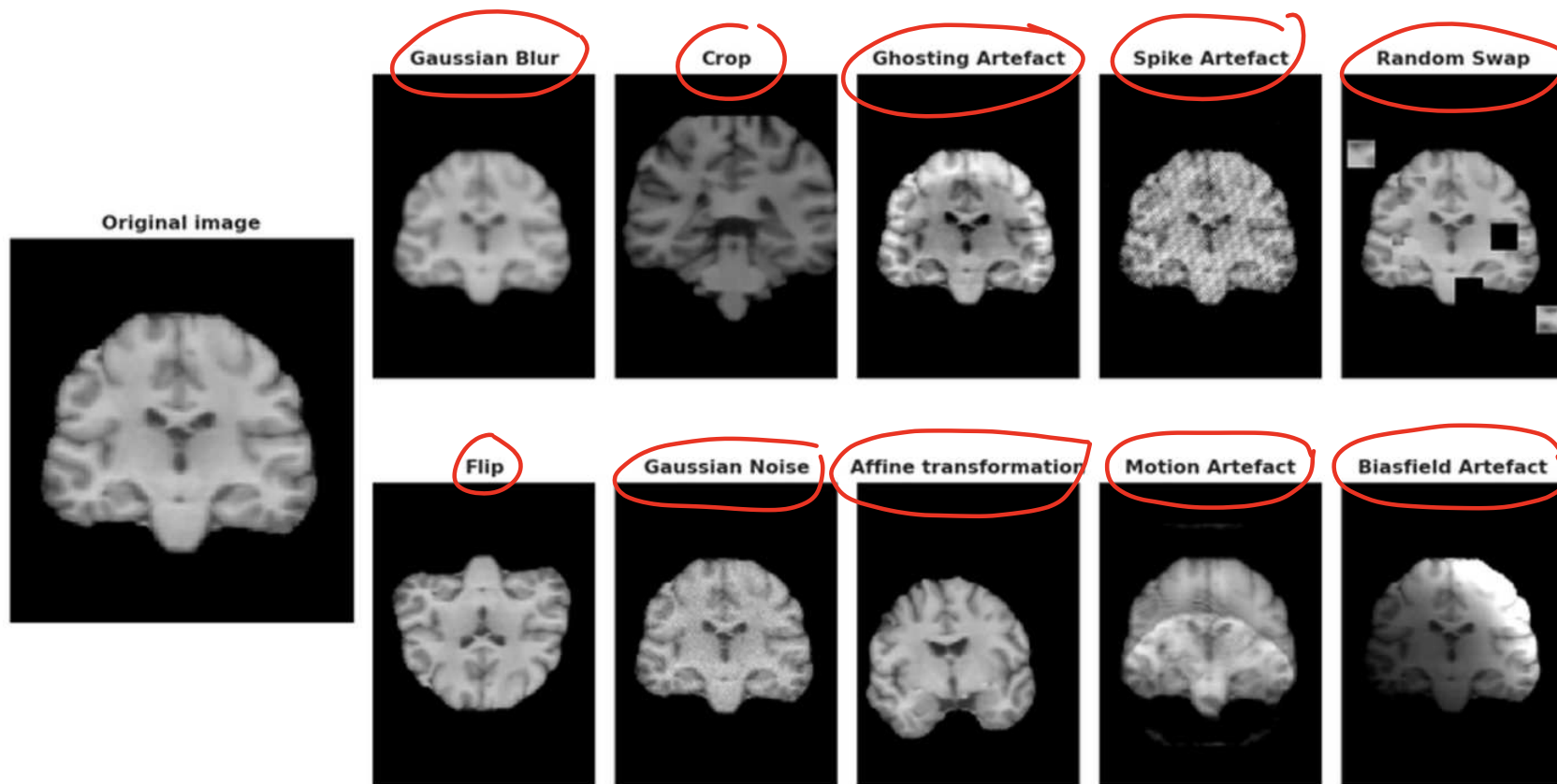
# Data Augmentation

a technique used in machine learning and deep learning to increase the diversity of data available for training models, without actually collecting new data.

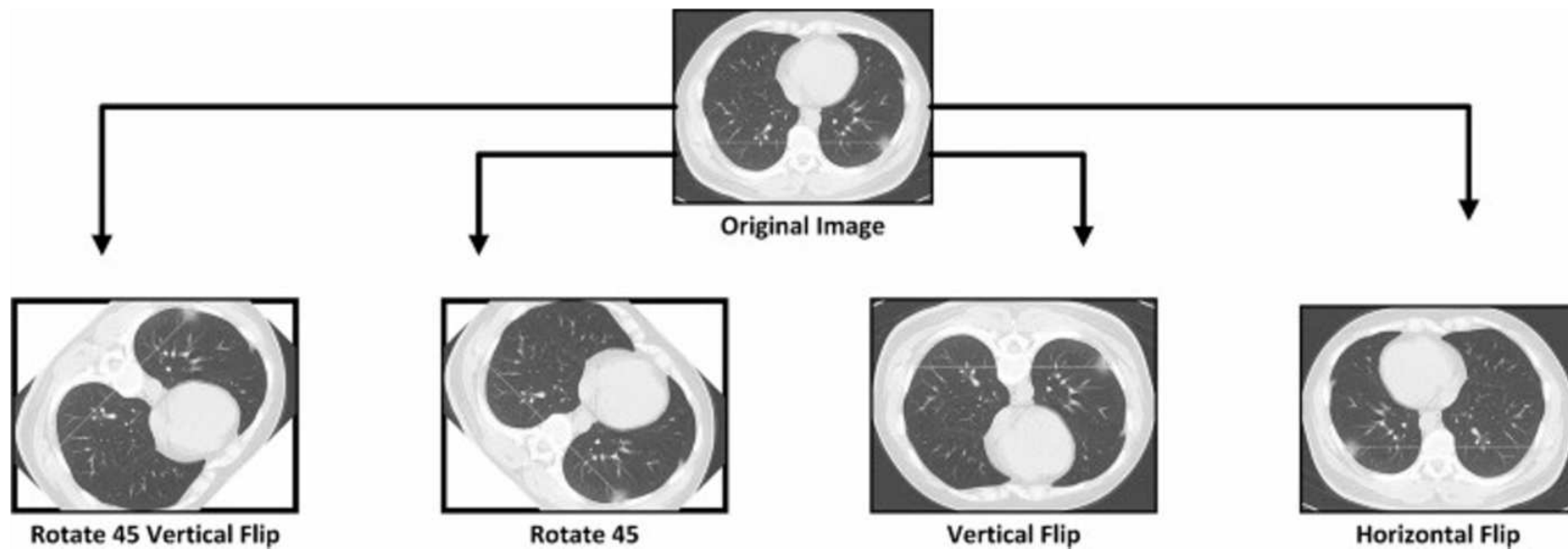
## **Benefits of Data Augmentation:**

- **Improved Model Performance:** Augmentation can lead to better generalization, reducing the risk of overfitting, especially in deep learning models that require large amounts of training data.
- **Handling Imbalanced Datasets:** It can help balance datasets by augmenting underrepresented classes.
- **Enhancing Robustness:** Models trained with augmented data can handle real-world variability (like different lighting conditions in images) more effectively.
- **Cost-Effective:** Augmentation provides a way to artificially expand datasets without the need and expense of actual data collection.

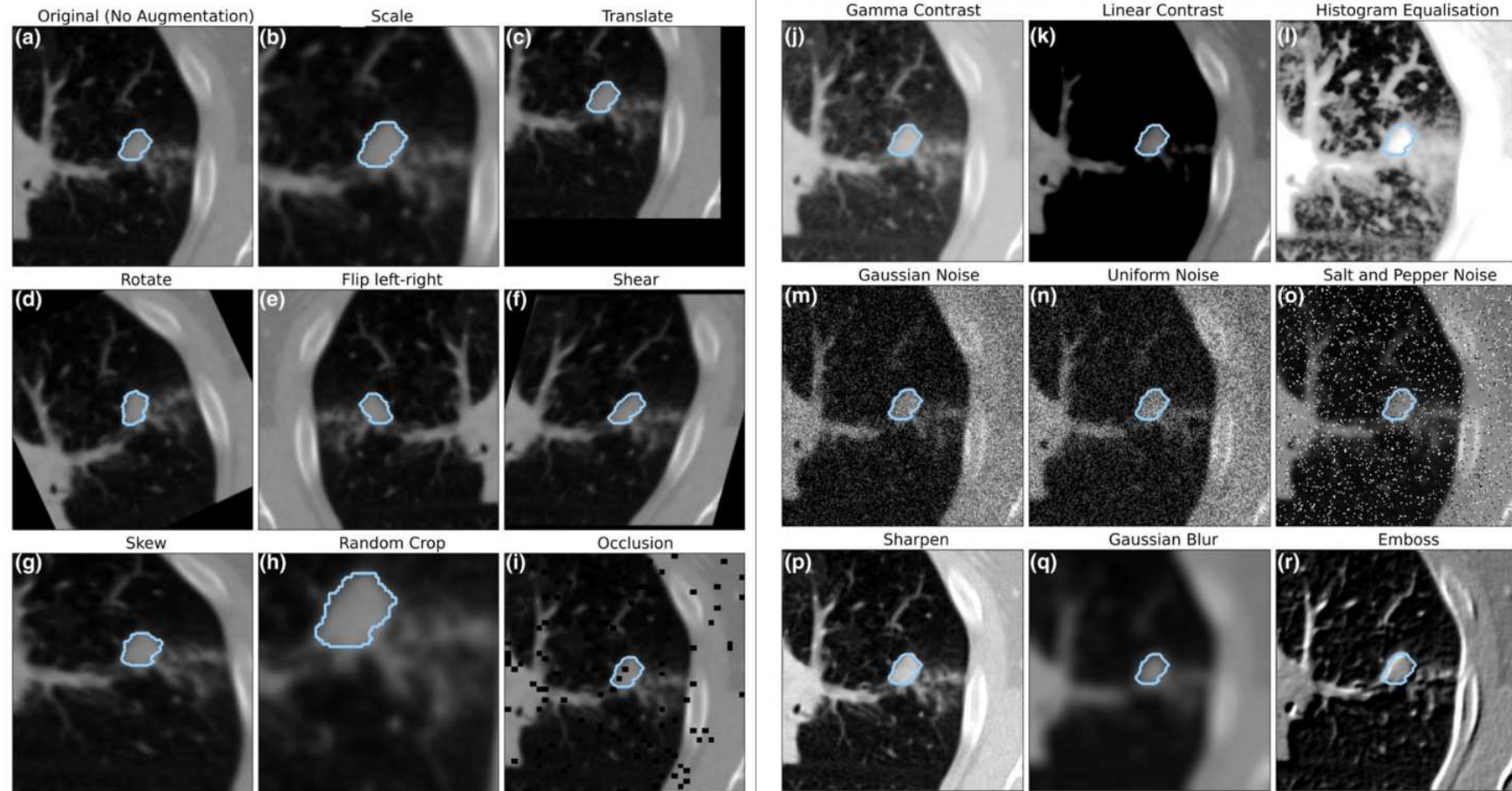
# Data Augmentation



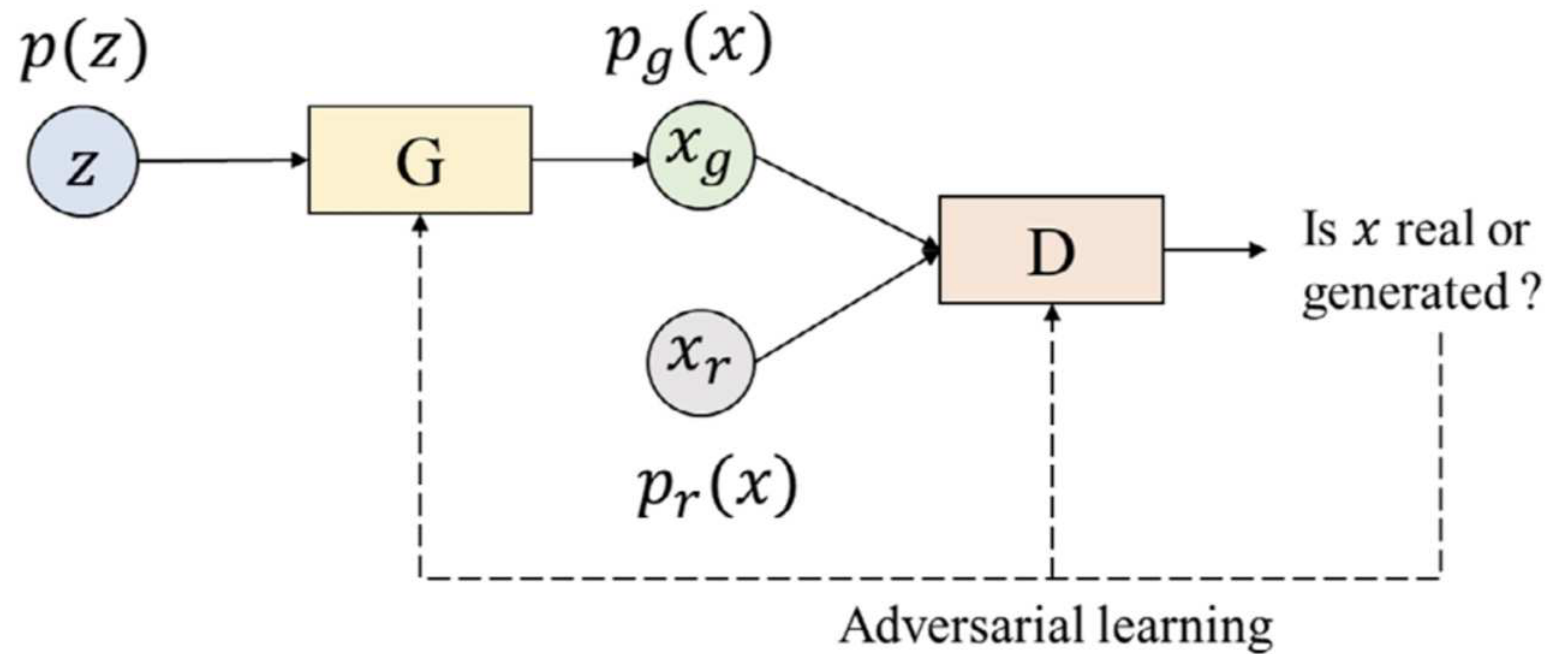
# Data Augmentation



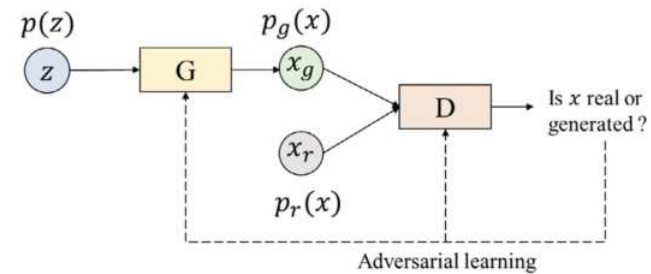
## Chlap et al 2021 – Review article



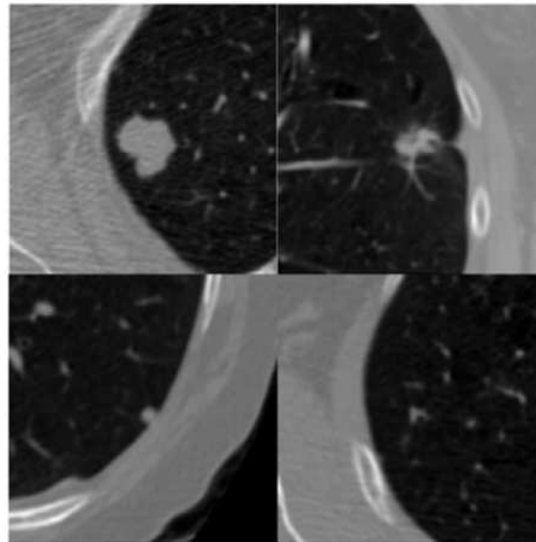
## Augmentation via Generative Models (\*)



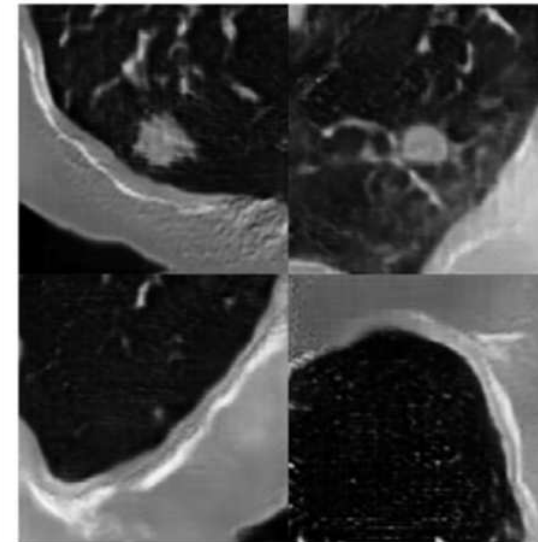
# Augmentation via Generative Models (lecture 8)



Real image



Synthetic image



## Data Augmentation - MixUp



60% cat

40% dog

<https://arxiv.org/pdf/1710.09412.pdf>

## Data Augmentation - MixUp



Sample illustration of a Mixup virtual instance comprised of two different classes. In this example, an fMRI scan of non-improver (left) and an fMRI scan of improver (middle) were linearly interpolated to form a third synthetic scan (right) with a soft label of 50% Improver and 50% Non-Improver. (improver → improve after 1 year treatment). [Smuchny et al NeuroImage 2022](#)

 **Key Insight:** MixUp **should NOT be used** when fine-grained texture information is critical (e.g., histopathology images).

 **Pro Tip:** Always test augmented models on **real, unmodified test sets** to avoid learning artifacts.



# Pipelines in code: four ways to say the same thing

*torchvision v2, Albumentations, Keras preprocessing layers, MONAI. Same concepts, different ergonomics: pick by ecosystem and by how much your task needs masks, boxes, or 3D.*

AUTHORED

Written for this course from the libraries' public documentation.

[docs.pytorch.org/vision](https://docs.pytorch.org/vision) · [albumentations.ai/docs](https://albumentations.ai/docs) · [keras.io](https://keras.io) · [docs.monai.io](https://docs.monai.io) (Project MONAI tutorials)

# Two pipelines, one contract

```
from torchvision.transforms import v2

train_tfm = v2.Compose([
    v2.RandomResizedCrop(224, antialias=True),
    v2.RandomHorizontalFlip(p=0.5),
    v2.ColorJitter(brightness=0.3, contrast=0.3),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(mean, std),
])

eval_tfm = v2.Compose([
    v2.Resize(256, antialias=True),
    v2.CenterCrop(224),           # deterministic
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(mean, std),     # same numbers
])
```

- **Random at train, deterministic at eval:** the EECS 498 pattern, as code. Validation accuracy measured through random transforms is noise.
- **Normalize appears in both** with identical constants; augmentation appears only on the left.
- Flips are not free everywhere: horizontal flip is wrong for chest X-rays with situs markers, chirality tasks, and text.
- Visual sanity check before training: **plot 8 augmented samples**; if a radiologist would object, so should you.

# When labels are pixels too

```
import albumentations as A

aug = A.Compose([
    A.RandomResizedCrop(size=(512, 512)),
    A.HorizontalFlip(p=0.5),
    A.CLAHE(p=0.3),
    A.RandomBrightnessContrast(p=0.3),
    A.Normalize(mean, std),
])

out = aug(image=img, mask=mask)
# same geometric transform hits both;
# masks are warped with nearest neighbor
```

- **Masks, bounding boxes, and keypoints move with the image:** the reason segmentation and detection work defaults to Albumentations.
- Rich photometric menu (CLAHE, gamma, blur, compression artifacts) that torchvision keeps minimal.
- **Licensing note for the startup side:** classic Albumentations is MIT; the successor AlbumentationsX is AGPL-3.0 / commercial dual-licensed. Check before shipping a product.

# Preprocessing that ships inside the model

```
from keras import layers, Sequential

model = Sequential([
    layers.Rescaling(1.0/255),
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.05),
    layers.RandomZoom(0.1),
    # ... the actual network ...
])
# augmentation runs only when training=True;
# at inference the layers pass images through
```

- The train/eval switch is **automatic**: the same exported model is honest at inference time by construction.
- Preprocessing is **saved with the model**, so deployment cannot drift from training: one fewer reproducibility gap.
- Runs on GPU inside the graph; heavy random pipelines stop stealing CPU from the data loader.

# Medical pipelines speak DICOM, spacing, and windows

```
from monai.transforms import *

pre = Compose([
    LoadImaged(keys=["image", "label"]),
    EnsureChannelFirstd(keys=["image", "label"]),
    Orientationd(keys=..., axcodes="RAS"),
    Spacingd(keys=..., pixdim=(1.5, 1.5, 2.0),
             mode=("bilinear", "nearest")),
    ScaleIntensityRanged(keys="image",
                        a_min=-57, a_max=164,      # CT window (HU)
                        b_min=0.0, b_max=1.0, clip=True),
])
```

Pattern from the MONAI spleen segmentation tutorial ([github.com/Project-MONAI/tutorials](https://github.com/Project-MONAI/tutorials)).

- **Dictionary transforms** keep image and label synchronized through every step, in 3D.
- Class 1's threads return as parameters: **voxel spacing** (`Spacingd`) and the **CT window** (`ScaleIntensityRanged` in Hounsfield units).
- Image resampled bilinearly, label with **nearest neighbor**: the segmentation-mask rule from class 1, enforced per key.

# Augmenting in 3D, scanner artifacts included

```
aug = Compose([
    RandCropByPosNegLabeld(keys=...,
        spatial_size=(96, 96, 96),
        pos=1, neg=1, num_samples=4),
    RandAffined(keys=..., prob=0.5,
        rotate_range=(0.1,)*3, scale_range=(0.1,)*3),
    RandGaussianNoised(keys="image", prob=0.2),
    RandBiasFieldd(keys="image", prob=0.2),
])

# TorchIO: the scanner-physics menu
import torchio as tio
physics = tio.Compose([tio.RandomMotion(),
    tio.RandomGhosting(), tio.RandomBiasField(),
    tio.RandomElasticDeformation()])
```

- Bias field, ghosting, motion: the artifacts Bagci's slides showed as **problems** become **training signal** when simulated on purpose.
- Patch-based cropping ( `RandCropByPosNegLabeld` ) balances foreground and background patches: 3D volumes rarely fit in memory whole.
- Keep augmentation **anatomically plausible**: a flipped heart is a diagnosis, not an augmentation.

# MixUp and CutMix live after the DataLoader

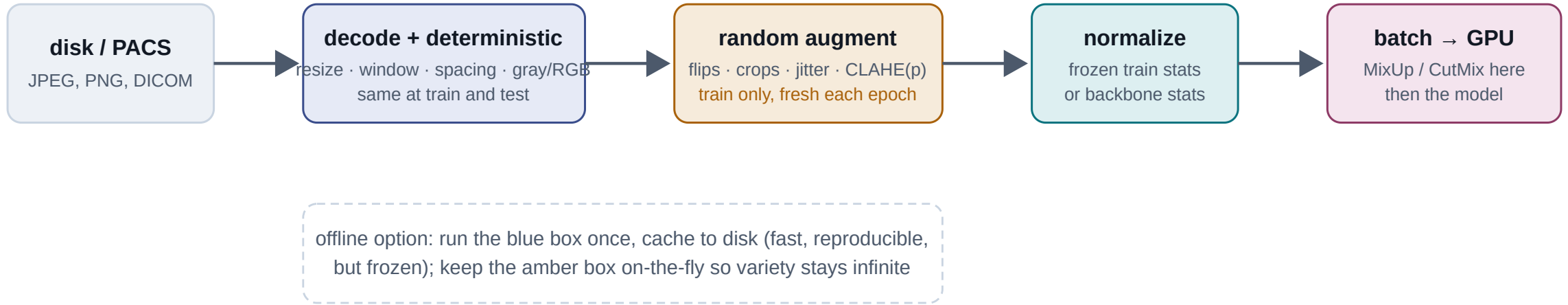
```
from torchvision.transforms import v2

mixer = v2.RandomChoice([
    v2.MixUp(num_classes=NUM_CLASSES),
    v2.CutMix(num_classes=NUM_CLASSES),
])

for images, targets in loader:
    images, targets = mixer(images, targets)
    # targets are now soft: cat 0.4 / dog 0.6
    loss = criterion(model(images), targets)
```

- These transforms need **pairs of samples**, so they run on the assembled batch, not per image: a different place in the pipeline than everything else today.
- Labels become **soft**; your loss must accept probability targets (cross entropy with label smoothing semantics).
- Johnson's tip: try Cutout and MixUp on small classification sets; Bagci's caution: skip MixUp where fine texture is the diagnosis, and **always evaluate on unmodified images**.

# Where each operation lives



## Reproducibility checklist:

seed the augmentation RNG (and DataLoader workers) · log every parameter · version the preprocessing code with the model  
and store the frozen normalization constants next to the checkpoint, never in a notebook cell

# If you remember nothing else

- 1. Classical filters are not obsolete: they are the vocabulary of your first CNN layer and of every artifact you will ever debug.
- 2. Every preprocessing statistic is fitted on the training split and frozen; medical data adds "and never across a patient boundary."
- 3. A pretrained backbone is a contract: its resize, its channel order, its normalization constants, or its benefit evaporates.
- 4. Augmentation is train-time randomness that must be averaged away, or switched off, at test time; evaluation always sees unmodified images.
- 5. A pipeline nobody can re-run is a result nobody can trust: seeds, parameters, and constants live in versioned code, not in memory.

# Your pipeline, end to end

- **Task.** Build one training pipeline for your project data in torchvision v2, Albumentations, or MONAI: deterministic preprocessing, at least three augmentations, honest eval transforms. Plot 8 augmented samples and write the one-paragraph methods description that would let a colleague reproduce it.
- **Leakage audit.** Point at the exact line where your normalization statistics are computed. Which split do they see? Patient-level safe?
- **Reading.** Shorten and Khoshgoftaar, augmentation survey, J. Big Data 2019 · Chlap et al., medical augmentation review, 2021 · MONAI tutorials repo · albumentations.ai/docs · fast.ai Practical Deep Learning (course.fast.ai).
- **Clinic connection.** Session 4's paper clinic question, "could you reproduce the preprocessing from the text alone?", now has teeth: apply it to your own draft.

# Source decks used in class 2

- 1 **EECS 498-007 / 598-005 Deep Learning for Computer Vision, Fall 2020, University of Michigan**, Justin Johnson. Lecture 10, Training Neural Networks, Part 1 (data preprocessing, data augmentation, regularization pattern).  
<https://web.eecs.umich.edu/~justincj/teaching/eecs498/FA2020/>  
© the author; shown unmodified for classroom teaching with attribution; not licensed for adaptation or redistribution.

---

- 2 **CS231n Convolutional Neural Networks for Visual Recognition, Spring 2017, Stanford University**, Fei-Fei Li, Justin Johnson, Serena Yeung. Lecture 7, Training Neural Networks, Part 2 (batch normalization recap, transfer learning).  
<https://cs231n.stanford.edu/> · slides: [cs231n.stanford.edu/slides/2017/](https://cs231n.stanford.edu/slides/2017/)  
Slide PDFs © the authors (no explicit license); the companion course notes at [cs231n.github.io](https://cs231n.github.io) are MIT-licensed and quoted on the rule slide.

---

- 3 **16-385 Computer Vision, Fall 2023, Carnegie Mellon University**, Ioannis Gkioulekas. Lecture 2, Image filtering.  
<http://16385.courses.cs.cmu.edu/fall2023/lectures>  
© Carnegie Mellon University and credited original authors; shown unmodified with attribution.

# Sources, continued

- 4 **BME 495 Deep Learning for Biomedical Imaging, Winter 2025, Northwestern University**, Ulas Bagci. Lectures 3–4 (image preprocessing, MRI intensity standardization after Nyul and Udupa) and Lecture 9 (data augmentation; review cited there: Chlap et al. 2021).  
<https://bagcilab.com/courses/> · publicly shared by the author; internal reference slides retained at each section's end.

---

- 5 **Library documentation** behind the authored code slides: torchvision transforms v2 ([docs.pytorch.org/vision](https://docs.pytorch.org/vision)), Albumentations ([albumentations.ai/docs](https://albumentations.ai/docs); AlbumentationsX is AGPL-3.0/commercial), Keras preprocessing layers ([keras.io](https://keras.io)), MONAI and its tutorials ([docs.monai.io](https://docs.monai.io), [github.com/Project-MONAI/tutorials](https://github.com/Project-MONAI/tutorials)), TorchIO ([torchio.readthedocs.io](https://torchio.readthedocs.io)), scikit-image, OpenCV.

---

- 6 **Course cross-references**: Kapoor and Narayanan, Leakage and the reproducibility crisis in ML-based science, Patterns 2023 (section 1.11) · nnU-Net, Isensee et al., Nature Methods 2021 (section 3.13) · stain normalization: Reinhard 2001, Macenko 2009, Vahadane 2016 (section 3.13) · OPG-SHAP cropping example (section 3.2).

---

- 7 **Compilation note.** Curated by T. Intharah for the Research Master's module Data Preparation for ML and AI, Khon Kaen University, August 2026. For classroom use; each source retains its own license.